

密 级： 公开

加密论文编号：

论文题目： 面向云原生应用权限提升漏洞的动态防
控技术研究

学 号： M202310652

研 究 生： 韩晓阳

专 业 名 称： 计算机科学与技术

2026 年 5 月 1 日

面向云原生应用权限提升漏洞的动态防控技术研究

**Research on Dynamic Prevention and Control
Technologies for Privilege Escalation
Vulnerabilities in Cloud-Native Applications**

研究生：韩晓阳

指导教师：孙昌爱

北京科技大学 计算机与通信工程学院

北京 100083，中国

Doctor Degree Candidate: Xiaoyang Han

Supervisor: Chang'ai Sun

School of Computer and Communication Engineering

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

中图分类号:

学校代码:

10008

U D C:

密 级:

公开

北京科技大学硕士学位论文

论文题目: 面向云原生应用权限提升漏洞的动态防控技术研究

研究生: 韩晓阳

指 导 教 师: 孙昌爱 单位: 北京科技大学 职称: 教授

指 导 小 组 成 员: 单位: 职称:

 单位: 职称:

论文提交日期: 2026 年 5 月 1 日

学位授予单位: 北 京 科 技 大 学

摘 要

论文中文摘要字数约为 300~600 字，如遇特殊需要字数可以略多，限一页。

论文摘要是论文内容不加注释和评论的简短陈述，一般以第三人称语气写成。摘要的编写应遵循下列原则：

1) 摘要应具有独立性和自含性，即不阅读论文的全文，就能获得必要的信息。摘要是学位论文的缩影，是学位论文的主要内容、见解、结论简短明了的缩写。

2) 摘要应是一篇完整的短文，可以独立使用，可以引用。

3) 摘要的内容应包含与论文等量的主要信息，供读者确定有无必要阅读全文，也可供文摘汇编等二次文献采用。

4) 摘要一般应说明研究工作的目的意义、研究方法、研究结果、主要结论及意义、创造性成果和新见解，而重点是结论和创新点。

5) 要用文字表达，不要附图和照片，除了实在无变通办法可用以外，摘要中不用图、表、化学结构式、非公知公用的符号和术语，不要使用表格、公式、上下标以及其他特殊符号，要突出重点，阐述清楚，少用数据表。论文摘要用语力求简洁、准确。原则上 300~600 字。

关键词：摘要；论文；要求；字数；格式（关键词个数为 3~5 个，正式写作请删除此括号）

Research on Dynamic Prevention and Control Technologies for Privilege Escalation Vulnerabilities in Cloud-Native Applications

ABSTRACT

In environmental economics, environmental resources including environmental quality are categorized as amenity resources. Due to its importance to human welfare, the amenity resources theoretical study and valuation is an ongoing issue at the academic frontier in the environmental economics area.

注：论文的英文摘要应有英文题目和关键词，内容与中文摘要相同，用另页置于中文摘要之后；外文摘要实词在 300 个左右。

Key Words: Key word 1; Key word 2; Key word 3; ...

序 (可选)

序并非必要。论文的序，一般是作者或他人对本篇基本特征的简介，如说明研究工作缘起、背景、主旨、目的、意义、编写体例，以及资助、支持、协作经过等；也可以评述和对相关问题研究阐述。这些内容也可以在主体部分引言中说明。

目 录

摘要	I
ABSTRACT	III
序 (可选)	V
1 引言	1
1.1 背景与意义	1
1.2 内容与成果	1
1.3 论文组织结构	1
2 背景介绍	3
2.1 相关概念与技术	3
2.1.1 容器	3
2.1.2 Kubernetes 架构与工作原理	4
2.1.3 Kubernetes 权限模型	5
2.1.4 Kubernetes 权限提升漏洞	6
2.2 云原生安全工具	7
2.2.1 容器运行时安全监控工具: Falco	7
2.2.2 准入控制与策略约束工具	8
2.2.3 网络隔离工具: NetworkPolicy	9
2.2.4 静态扫描与配置审计工具: 镜像、依赖与 IaC	10
2.2.5 AI Agent 技术	11
2.3 国内外研究现状	12
2.4 动机与挑战	13
3 面向云原生权限提升漏洞的智能防控技术	15
3.1 形式化定义	15
3.1.1 基本对象	15
3.1.2 评价与选择	16
3.2 总体框架	17
3.3 开源应用漏洞安全知识库构建	18
3.3.1 漏洞收集与分类框架	18
3.3.2 知识库构建	21
3.4 基于多智能体协同的策略生成与优化	21
3.4.1 安全上下文构建	22
3.4.2 基于漏洞分析思维链的策略生成	23
3.4.3 基于评审反馈的策略优化	28

3.5 策略评估与质量控制	32
3.5.1 评价算子	32
3.5.2 多智能体评估体系	34
3.5.3 多源排名聚合模型	35
4 面向云原生权限提升智能化漏洞防控系统工具	37
4.1 需求分析	37
4.2 系统设计	38
4.3 系统实现与演示	40
4.3.1 核心模块	40
4.3.2 证据与知识库服务	40
4.3.3 策略生成与优化服务	41
4.3.4 质量评价与审阅	41
4.3.5 策略发布与回滚	42
5 实验研究	43
5.1 研究问题	43
5.2 实验设置	43
5.3 RQ1: 防控技术 workflow 整体优势	44
5.3.1 三维度总体表现	44
5.3.2 有效性维度的消融贡献与稳定性	44
5.3.3 样本层面差异与离群	46
5.4 RQ2: 跨模型稳健性	46
5.4.1 生成模型分层	46
5.4.2 评审模型与三维权衡	48
5.4.3 模型内容对其能力评估	48
5.5 RQ3: 评价一致性与可信度	50
5.6 RQ4: 静态扫描的处置价值	51
5.7 RQ5: 端到端资源消耗与效率评估（令牌消耗）	52
6 案例分析	55
6.1 案例选择与实验设计	55
6.1.1 CVE 案例选择标准	55
6.1.2 实验环境与复现流程	55
6.2 案例一：CVE-2022-24768 内部权限问题	57
6.2.1 漏洞详解与复现	57
6.2.2 策略部署	58
6.2.3 三因素分析	59

6.3 案例二：CVE-2025-2241 (Hive ClusterProvision 敏感信息泄漏)	60
6.3.1 漏洞详解与复现	60
6.3.2 策略部署	62
6.3.3 三因素分析	62
6.4 案例三：CVE-2023-30512 (CubeFS CSI 插件集群权限提升)	63
6.4.1 漏洞详解与复现	63
6.4.2 策略部署	66
6.4.3 三因素分析	66
6.5 案例四：CVE-2020-4062 Conjur OSS Helm Chart 数据库网络暴露	67
6.5.1 漏洞详解与复现	67
6.5.2 策略部署	68
6.5.3 三因素分析	69
7 结论与展望	71
7.1 结论	71
7.2 展望	71
附录 A 单位	77
附录 B CVE-2022-24768 复现步骤与策略清单	79
B.1 漏洞复现详细步骤	79
B.2 策略代码清单	81
B.2.1 Kubernetes RBAC: 收敛 AppProject 的写权限	81
B.2.2 Argo CD RBAC: 默认只读 + 高风险能力隔离	83
B.2.3 准入控制: Gatekeeper (推荐)	84
B.2.4 准入控制: Kyverno (替代方案)	88
附录 C CVE-2025-2241 复现步骤与策略清单	93
C.1 漏洞复现详细步骤 (受控测试)	93
C.2 策略代码清单	94
C.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)	95
C.2.2 2) 对读取行为启用审计并联动告警	95
C.2.3 3) 缩短暴露窗口: 供应完成后清理 ClusterProvision	96
C.2.4 4) vCenter 凭据轮换与强化 (止血)	99
C.2.5 5) “敏感字段守护”检测 (Gatekeeper/Kyverno, 建议审计模式)	100
附录 D CVE-2023-30512 复现步骤与策略清单	105
D.1 漏洞复现详细步骤 (验证性复现)	105
D.2 策略代码清单	106
D.2.1 1) RBAC 最小化: 移除 ClusterRole 中的 secrets 权限	106

D.2.2 2) ServiceAccount 硬化 (按需评估)	107
D.2.3 3) 准入防回归 (OPA Gatekeeper): 禁止创建包含 secrets 读取动词的 ClusterRole	108
D.2.4 4) 审计与检测: 对 secrets 枚举行为进行可观测化	109
D.2.5 5) 网络收敛 (可选): 限制 CSI Pod 出站访问面	110
附录 E CVE-2020-4062 复现步骤与策略清单	113
E.1 漏洞复现详细步骤: 受控测试	113
E.2 策略代码清单	114
E.2.1 1) 根本修复: 升级 Chart	114
E.2.2 2) 缓解措施: NetworkPolicy, 默认拒绝并仅放通 Conjur Server 访问 Postgres 5432	114
E.2.3 3) 命名空间隔离与 RBAC 收敛	115
致谢	117
作者简历及在学研究成果	119
独创性说明	121
关于论文使用授权的说明	123
学位论文数据集	125

1 引言

1.1 背景与意义

Kubernetes 已成为云原生基础设施的事实标准，并被广泛用于承载容器化应用的部署、调度与弹性伸缩等关键能力 [1-3]。随着云原生生态不断扩展，大量第三方组件（监控、服务网格、CI/CD 等）以 Sidecar、Controller 或 Operator 等形式接入集群，其运行往往依赖 ServiceAccount 与 RBAC 权限配置来访问或操作集群资源。由于权限边界复杂、配置高度异构且随业务迭代频繁变更，权限配置不当容易引入过度授权与越权访问风险，进一步触发敏感资源泄露或未授权操作等安全事件。

从权限视角看，Kubernetes 基于扩展 RBAC 机制对资源访问进行统一管理 [1, 4]，而最小权限原则则为权限配置治理提供了通用的安全基线 [5]。然而，在实际工程中，第三方应用的权限需求往往难以被精确刻画，导致“为保证可用而放宽权限”的做法普遍存在，从而产生冗余权限并扩大攻击面 [6, 7]。与此同时，权限提升攻击还可能借助配置缺陷、DevOps 流水线以及运行时行为实现权限扩张，使得仅依赖静态规则的检测方法在动态环境下容易出现适应性不足与精度受限等问题 [8, 9]。

在漏洞响应层面，高危漏洞的补丁发布、版本验证与生产升级通常存在时间差；当补丁不可用或难以及时部署时，生产环境需要先采用可快速下发、可验证且可回滚的临时防控措施，以尽可能降低风险暴露面并抑制攻击链扩散。基于上述现实需求，本文面向云原生应用权限提升漏洞，聚焦在补丁窗口期内的动态防控：以漏洞公告和证据可追溯的上下文为输入，自动生成并筛选可交付的临时防控策略，从而提升集群的抗攻击能力。

1.2 内容与成果

为应对云原生权限提升漏洞的临时防控需求，本文主要工作与贡献如下：

1.3 论文组织结构

全文结构如下：第2章介绍相关概念、技术与国内外研究现状；第3章给出面向云原生权限提升漏洞的智能防控技术体系；第4章描述漏洞防控系统

设计与实现；第**5**章呈现实验设置与结果；第**6**章给出案例分析；第**7**章总结与展望。

2 背景介绍

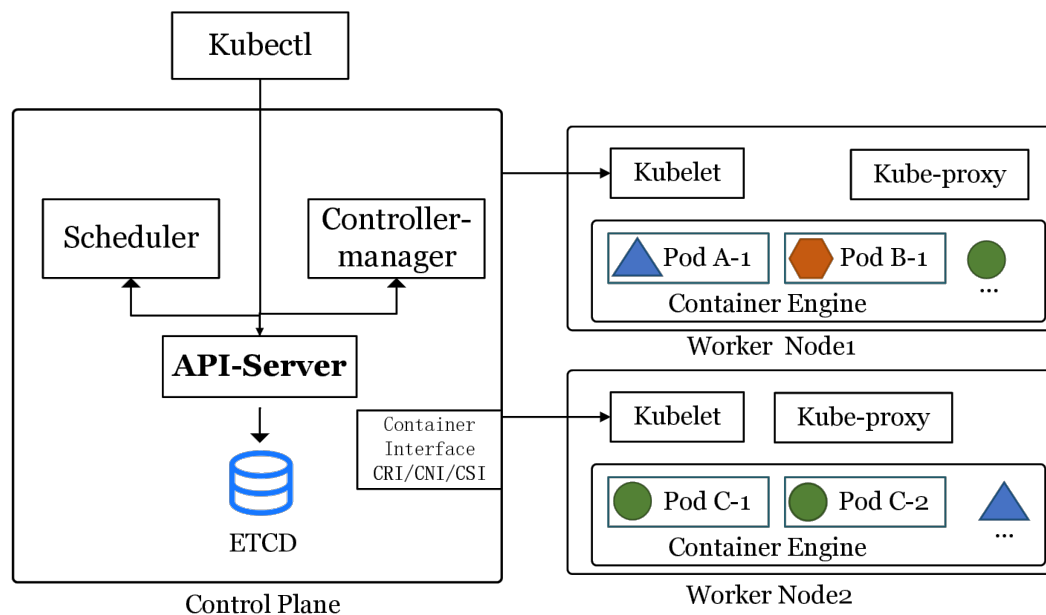
2.1 相关概念与技术

2.1.1 容器

容器（Container）是一种以**镜像（Image）**为交付单元、以操作系统内核能力为隔离基础的轻量级运行环境。与传统虚拟机通过虚拟化硬件来隔离不同，容器更强调**进程级隔离与环境一致性**：应用及其依赖以镜像形式打包交付，运行时由容器引擎创建隔离的进程视图与资源配额，再启动容器内的目标进程。

从实现机制看，容器的关键能力主要来自三类内核与运行时支撑：**（1）命名空间（Namespace）隔离**，将进程、网络、挂载点、IPC、用户等资源视图隔离开来，使容器内进程“看见”的系统环境与宿主机相互独立；**（2）控制组（cgroup）资源管控**，对 CPU、内存、I/O 等资源进行限制与统计，防止单个工作负载挤占宿主机资源；**（3）镜像与分层文件系统**，将应用与依赖固化为可复用的只读层，并基于 OverlayFS 技术叠加读写层形成运行时文件系统，从而同时实现复用、缓存与快速分发。

在工程实践中，容器镜像的可复用性与快速分发构成云原生交付的关键能力，但亦伴随若干显著的安全隐患。首先，镜像可能包含过期依赖或不当默认配置。长期未更新的基础镜像会导致已知漏洞残留；容器若以 `root` 身份运行，或在镜像中嵌入调试工具与敏感凭据，则会扩大被利用的风险。其次，容器的隔离能力取决于宿主机内核与运行时配置。启用特权模式、挂载宿主机路径、放宽 Linux capabilities，或禁用 `seccomp` 与 `AppArmor` 等安全机制，均会削弱容器与宿主机之间的隔离，从而使攻击面从容器内部向宿主机乃至整个集群扩展。鉴于上述风险，容器安全治理应覆盖镜像构建阶段的依赖管理与基线加固、制品交付环节的制品管理与签名，以及运行时的权限最小化与最小暴露等生命周期环节，并在各阶段引入自动化检测与策略约束以降低总体风险。



Architecture of the Kubernetes

图 2-1: Kubernetes 体系结构示意

2.1.2 Kubernetes 架构与工作原理

Kubernetes 是面向容器工作负载的编排系统,其核心思想是以**声明式 API**描述期望状态 (desired state),再由控制平面持续驱动实际状态 (actual state)向期望状态收敛 [1]。这一“**控制回路 (reconcile loop)**”使得部署、扩缩容、故障自愈与滚动升级等运维动作可以自动化、可重复且具备可观测性。

从整体架构看,Kubernetes 由**控制平面 (Control Plane)**与**工作节点 (Worker Node)**两部分构成。控制平面负责提供 API、调度与集群控制逻辑,其中:**API Server**是所有资源访问与状态变更的统一入口,承担认证、授权、准入与资源对象持久化等关键职责;**调度器 (Scheduler)**根据资源请求、亲和/反亲和、污点与容忍等约束为待调度 Pod 选择节点;各类**控制器 (Controller)**(如 Deployment/ReplicaSet、Job、Node、Endpoint 等控制器)基于资源对象的当前状态触发协调动作,形成“观察—决策—执行”的闭环。工作节点侧,kubelet 负责与 API Server 通信并将 Pod 规格落实到本机运行时,容器运行时 (Container Runtime) 负责拉取镜像、创建容器并管理生命周期,kube-proxy (或由 CNI/eBPF 方案替代)负责服务转发与网络规则的实现。如图2-1所示给出了 Kubernetes 的体系结构示意。

围绕上述组件,Kubernetes 进一步抽象出一组核心对象以承载不同层面的运维语义:**Pod**是最小调度单元,包含一个或多个共享网络与存储命名

空间的容器；**Deployment/StatefulSet/DaemonSet** 分别面向无状态、有状态与节点守护型工作负载提供期望副本与更新策略；**Service** 与 **Endpoint/EndpointSlice** 提供稳定的服务发现与负载均衡抽象；**Ingress**（或 Gateway API）面向南北向流量提供入口控制；**ConfigMap/Secret** 用于配置与敏感信息的声明式注入；**Volume/PV/PVC** 则提供存储生命周期与绑定关系的抽象。

Kubernetes 的工作原理可以概括为“资源对象驱动的状态机”。当用户提交 YAML 清单或通过控制器创建资源对象时，请求首先进入 API Server：经过认证、授权与准入控制后，资源对象被持久化并成为集群事实状态。随后，控制器与调度器通过监听资源变化（watch）获取事件：调度器为 Pending 的 Pod 选择节点并写回绑定结果；对应节点上的 kubelet 读取 PodSpec，调用容器运行时拉取镜像、创建容器，配置网络与挂载卷，并持续上报状态与探针结果。一旦发生节点故障、容器退出或副本数变化，控制器会再次触发协调动作，保证系统最终收敛到期望状态。

2.1.3 Kubernetes 权限模型

Kubernetes 将“资源访问”统一收敛到 API Server，使得安全治理的关键落在身份（Authentication）与授权（Authorization）链路之上。其中，RBAC（Role-Based Access Control）是最核心的授权模型之一 [1, 4]。在典型流程中，工作负载（Pod）以 ServiceAccount 作为身份主体；权限以 Role/ClusterRole 的形式被抽象为若干规则（rules），规则通常以“API 组与资源类型（apiGroups/resources）—操作动词（verbs）—作用域（namespace/cluster）”等维度描述允许的操作集合；随后通过 RoleBinding/ClusterRoleBinding 将主体与角色绑定，使得主体获得对特定资源的访问能力。与之配合，Namespace 用于隔离租户与资源域，NetworkPolicy 等机制用于收敛网络可达性并减少横向移动空间。

在落地层面，RBAC 通常以 YAML 清单进行声明式配置：通过编写 Role/ClusterRole 的规则集合，再以（Cluster）RoleBinding 将其授予 ServiceAccount 等主体。图2-2给出了权限配置（YAML）的示意，用于直观展示“规则—绑定—主体”的授权链路。

在工程实践中，第三方组件（如 Prometheus、Jenkins、Istio 等）接入往往需要访问多类资源与 API，对权限配置的依赖更强；一旦存在过宽的 RBAC 绑定或错误的身份边界（例如将集群级角色绑定到不必要的 ServiceAccount），

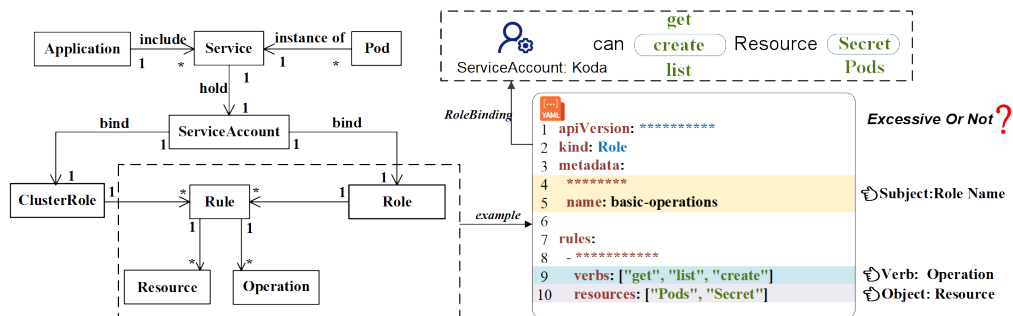


图 2-2: Kubernetes RBAC 权限配置 (YAML) 示意 [10]

便可能形成权限滥用与权限提升的入口。因此，理解从“主体—绑定—角色规则—资源对象”的授权链路，以及其与命名空间隔离、网络可达性的耦合关系，是后续分析权限提升漏洞成因与制定可落地防控措施的基础。

2.1.4 Kubernetes 权限提升漏洞

权限提升（Privilege Escalation）指攻击者在仅具备受限权限或初始执行能力的情况下，借助漏洞或错误配置，将自身权限提升到更高等级的过程。在 Kubernetes 场景下，权限提升既可能表现为授权范围扩大（例如能够访问更多 Kubernetes API，这通常由 RBAC 规则决定），也可能表现为节点侧系统权限提升（例如获得节点上的 root 权限），从而对更大范围的资源与工作负载产生影响。

在云原生环境中，权限提升的危害往往具有更强的放大效应。一方面，单个节点通常承载多个 Pod，节点侧提权成功后，攻击者可能对同节点上的多个工作负载实施窃取敏感信息、篡改运行环境或注入恶意组件等行为。另一方面，Kubernetes 的资源对象与运维动作高度集中且自动化；当攻击者获得更高权限后，可能进一步读取 Secret、创建高权限工作负载、修改网络策略或部署守护进程，进而将影响从单一应用扩展到集群层面的安全与稳定。因此，权限提升漏洞不仅是单点缺陷，还可能引发平台级安全事件与隔离失效。

为说明节点侧提权在云原生场景中的现实危害，图2-3给出了 CVE-2024-33522 的示例。该漏洞影响 Calico 的 CNI 安装程序：在部分易受影响版本中，安装二进制的 SUID（Set User ID）位配置不当，同时允许攻击者控制输入二进制。在攻击者已具备 Kubernetes 节点本地访问的前提下，攻击者可诱导安装过程以更高权限执行任意二进制，从而实现节点侧权限提升。虽然该漏洞的利用前提包含节点本地访问，但在真实环境中，这一前提可能由其他入侵路径满足（例如节点服务暴露、弱口令、供应链投毒，或通过其他漏洞获得

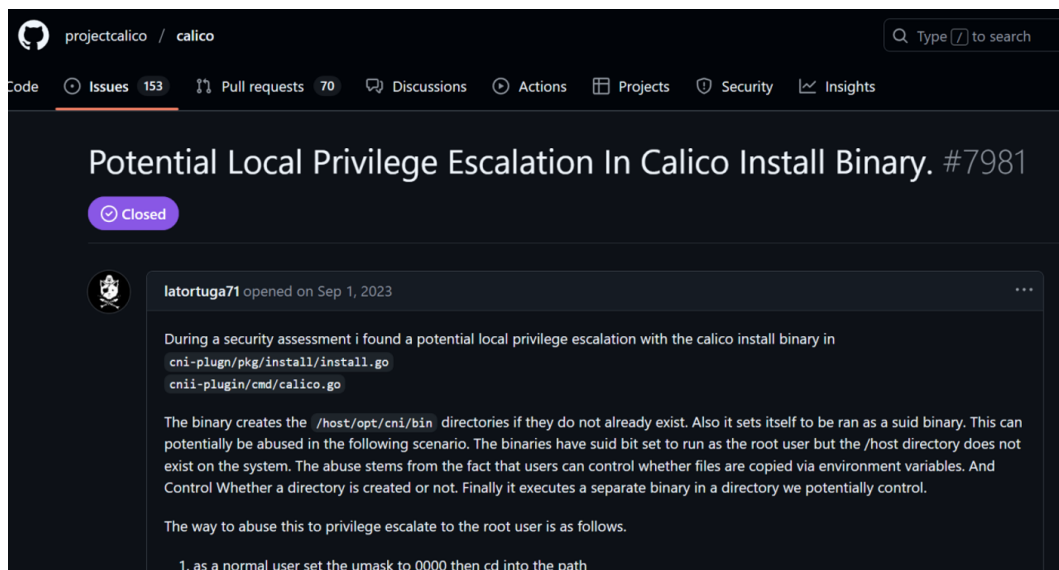


图 2-3: CVE-2024-33522 (Calico) 节点侧权限提升漏洞报告 ISSUE

节点执行能力)。一旦节点侧权限提升成功，攻击者可能读取或篡改节点关键数据（如容器运行时目录、配置与日志），窃取工作负载凭据与令牌，干扰网络组件行为，并进一步扩大影响范围，从而显著加剧集群整体风险。

2.2 云原生安全工具

在补丁未就绪或难以及时升级的阶段，生产环境往往需要依赖可快速下发、可验证且可回滚的控制手段来降低风险暴露面。从“上线前治理”到“运行时防护”的完整链路来看，云原生安全工具通常覆盖四个互补层面：（1）运行时检测与审计（在不中断业务的前提下对异常行为进行告警与取证）；（2）准入与策略约束（在资源进入集群前阻断高风险配置）；（3）网络隔离与最小连通（收敛工作负载间可达性以抑制横向移动）；（4）静态扫描与配置审计（在构建与交付阶段发现已知漏洞与弱配置）。下文分别介绍运行时检测工具 Falco、准入控制工具 OPA/Gatekeeper 与 Kyverno、网络隔离机制 NetworkPolicy，以及静态扫描与配置审计类工具。

2.2.1 容器运行时安全监控工具：Falco

在容器技术与云原生基础设施广泛部署的背景下，传统基于主机或网络的入侵监测系统（IDS）逐渐暴露出适应性不足、粒度较粗以及缺乏云原生上下文语义等问题。为满足容器化环境对运行时安全的实时性与上下文感知需求，开源项目 Falco 应运而生。Falco 最初由 Sysdig 公司开发，并于 2018 年捐赠给云原生计算基金会（CNCF），成为较早面向容器运行时的行为监测系

统之一。其核心思想是在**系统调用层**进行透明监控，并通过规则语言定义安全策略，从而在不中断业务的前提下实现精细化威胁监测与审计。

Falco 的监测机制基于 **eBPF** 技术对 Linux 系统调用的动态捕获与分析。**eBPF**（extended Berkeley Packet Filter）允许在**内核态**运行经过验证的字节码程序，并以受控方式访问内核提供的上下文与辅助函数；**eBPF** 程序可动态挂载到系统调用入口/退出点、内核跟踪点（tracepoint）、kprobe 以及网络相关钩子等位置，从而以较低侵入性捕获安全相关事件。相较于传统内核模块方案，**eBPF** 通过验证器与隔离执行机制降低了对内核稳定性的影响，并在性能与可观测性之间提供更可控的权衡，适合用于容器运行时的持续监测。

基于高语义的行为数据流，Falco 利用面向安全运维的领域特定语言（DSL）构建声明式规则引擎。与依赖统计特征或黑盒模型的检测方法不同，Falco 允许管理员针对特定威胁场景定义确定性的逻辑约束。例如图2-4展示了监测“在容器内启动交互式 shell 进程”的 Falco 规则，一旦检测到会产生日志/告警。这种白盒化的检测方式不仅具备极强的可解释性，便于合规审计与取证分析，还能确保告警信息直接关联至具体的云原生实体，显著降低了安全运营的定位成本。然而，该机制在工程应用中仍存在局限性：一方面，基于规则的检测高度依赖专家经验与知识库的持续更新，对未知攻击或变种攻击的泛化检测能力较弱；另一方面，Falco 的观测视野局限于系统调用层，对应用层协议滥用或纯内存型攻击的感知能力有限。

```
1 - rule: Terminal Shell in Container
2   desc: Detects the spawning of a shell process inside a container.
3   condition: >
4     evt.type = execve and
5     container.id != host and
6     proc.name in (bash, sh, zsh)
7   output: "Notice: Shell spawned (user=%user.name container_id=%container.
8     id container_name=%container.name cmd=%proc.cmdline)"
9   priority: WARNING
10
```

图 2-4: Falco 运行时监控与规则示意

2.2.2 准入控制与策略约束工具

准入控制（Admission Control）作为云原生安全的事前防护机制，通过拦截 API Server 请求，在资源对象持久化之前对其进行校验（Validation）或变更（Mutation），从而从源头阻断不合规配置进入集群。在这一领域，OPA（Open Policy Agent）及其 Kubernetes 实现 Gatekeeper 提供了通用的策略引擎

支持，但其依赖 Rego 语言的特性在一定程度上提高了使用门槛。

相比之下，**Kyverno** 作为 Kubernetes 原生策略引擎，通过以 YAML 定义策略降低复杂度。Kyverno 不仅支持对 Pod 安全标准（PSS）、镜像签名及供应链安全的校验，还具备对资源进行自动修正（Mutate）与基于触发器生成配置（Generate）的能力，能够将安全基线与平台治理要求固化为可复用的声明式规则。如图 2-5 所示，该策略展示了一个典型的 Kyverno 应用场景：强制要求所有 Pod 必须携带 owner 标签。通过定义 `validationFailureAction: enforce`，任何缺失该元数据的 Pod 创建请求都将被准入控制器直接拒绝。这种声明式的策略管理方式不仅实现了“所见即所得”的治理效果，更确保了集群内所有工作负载具备可追溯的权属标识。在漏洞补丁尚不可用时，此类

```
demo > kyverno
1  apiVersion: kyverno.io/v1
2  kind: ClusterPolicy
3  metadata:
4    name: audit-env-label
5  spec:
6    validationFailureAction: audit # <--- 关键点：这里是 audit（审计），而不是 enforce（强制）
7    background: true              # 对集群里已经存在的 Pod 也进行扫描
8    rules:
9      - name: check-env-label
10       match:
11         resources:
12           kinds: [Pod]
13       validate:
14         message: "建议添加 env 标签以区分环境。"
15         pattern:
16           metadata:
17             labels:
18               env: "?*" # "?*" 意思是有这个 key 且 value 不为空
```

图 2-5: Kyverno 策略示例：强制要求 Pod 包含 owner 标签

准入策略尤其适用于**快速止血**（Virtual Patching）：通过强制最小权限、限制高危特性（如 privileged 容器）与收敛敏感对象访问边界，可显著压缩攻击面。同时，准入策略的发布与回滚成本较低，便于安全团队在不同业务环境中进行灰度验证与影响评估，构建起灵活且坚固的配置安全防线。

2.2.3 网络隔离工具：NetworkPolicy

网络隔离用于降低横向移动与数据外传风险，是云原生环境中与身份权限控制互补的关键手段。Kubernetes 的 NetworkPolicy 为声明式策略，允许以 Pod/Namespaced 选择器与端口规则的形式，定义入站（Ingress）/出站（Egress）通信的允许集合，从而将网络访问从“默认全通”收敛到“默认拒绝 + 按需放通”。在多租户或微服务体系中，合理的 NetworkPolicy 可将控制域落到命名空间与工作负载级别，减少单点被攻破后对整个集群的扩散影响。

需要注意的是，NetworkPolicy 的效果依赖底层网络插件（CNI）对策略的

实现能力。常见 CNI（如 Calico、Cilium、Antrea 等）均可提供对 NetworkPolicy 的支持；其中部分实现还可在标准三/四层策略之外，扩展更细粒度能力（例如基于 eBPF 的高性能转发与可观测性、面向应用协议的七层策略等）。工程实践中通常结合业务依赖关系梳理通信图谱，先对关键命名空间与高价值服务进行最小连通收敛，再逐步扩展覆盖范围，并配合观测与告警机制验证策略变更带来的连通性影响。

```
YAML

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-frontend-only
spec:
  # 对应描述: “针对带有 role: db 标签的数据库工作负载”
  podSelector:
    matchLabels:
      role: db

  # 声明策略类型为入站控制
  policyTypes:
    - Ingress

  ingress:
    # 对应描述: “仅允许来自 role: frontend 前端服务的流量”
    - from:
        - podSelector:
            matchLabels:
              role: frontend

    # 对应描述: “通过 TCP 5432 端口访问”
    ports:
      - protocol: TCP
        port: 5432
```

图 2-6: NetworkPolicy 示例：限制数据库仅允许前端服务访问

如图 2-6 所示，该策略定义了一个典型的微服务网络隔离场景：针对带有 `role: db` 标签的数据库工作负载，仅允许来自 `role: frontend` 前端服务的流量通过 TCP 5432 端口访问。这种基于标签选择器（Label Selector）的白名单机制有效地构建了微隔离（Micro-segmentation）边界，确保即使集群内其他容器（如被入侵的非关联服务）试图横向移动访问数据库，也会在网络层被 CNI 插件直接阻断，从而显著提升了核心数据资产的安全性。

2.2.4 静态扫描与配置审计工具：镜像、依赖与 IaC

与运行时检测相对，静态扫描更偏向事前发现与持续治理：在镜像构建、交付与部署前，通过自动化检查提前识别已知漏洞、弱配置与合规风险，避免高风险对象进入集群。

第一类是**镜像与依赖漏洞扫描**。该类工具通常解析镜像分层文件系统与软件包清单，基于公开漏洞库对已知 CVE 进行匹配，并输出风险等级、受影响版本与修复建议。代表性工具包括 Trivy[11]、Grype/Anchore[12] 等；其中，Syft[13] 等工具可生成软件物料清单（SBOM），用于提升漏洞告警的可追溯性与供应链治理能力。在工程实践中，镜像扫描往往与 CI 流水线结合，在构建阶段阻断高危漏洞或不合规组件进入制品仓库。

第二类是**Kubernetes 清单与基础设施即代码（IaC）扫描**。此类工具面向 YAML/Helm/Terraform 等配置，检查特权容器、宿主机路径挂载、过宽权限、明文密钥、缺失资源限制等常见风险，并可作为代码评审与准入策略的前置门禁。例如，kubelinter[14]、kubescape[15]、kube-score[16]、kubesec[17]、Polaris[18] 等可对工作负载清单进行规则化检查；Checkov[19] 等可对 Terraform 等 IaC 模板进行安全基线与合规性验证。静态扫描的优势在于易于规模化推广与持续运行，但也可能因业务差异产生误报，因此通常需要结合组织基线与例外机制、以及准入与运行时告警进行闭环。

2.2.5 AI Agent 技术

AI Agent（智能体）是以大语言模型为核心、能够**面向任务进行分解与规划**并通过**调用外部工具/系统**完成闭环执行的智能系统形态。与仅进行对话式问答的模型不同，Agent 更强调“**感知—决策—执行—反馈**”的迭代过程：其输出不仅包括自然语言解释，也可包含结构化计划、可执行命令、配置片段或对外部 API 的调用，从而在复杂场景中实现可复用的自动化能力。

从实现要素看，AI Agent 往往由以下模块组合构成：**（1）任务规划与状态管理**，将目标拆分为若干可验证步骤，并在执行过程中维护中间状态与失败回退；**（2）记忆与知识增强**，通过短期记忆记录上下文，通过长期记忆或外部知识库（如检索增强生成，RAG）获取领域知识与证据；**（3）工具调用与执行器**，将模型推理结果转化为对外部工具的调用（例如代码生成、静态分析、配置校验、查询检索等），并把工具输出反馈给模型以驱动下一轮修正；**（4）安全与约束机制**，对可执行操作施加权限边界、审计记录与输出格式约束，以降低误操作风险。

在工程实践中，工具调用通常以标准化协议或接口实现，以便将不同能力（如配置语法检查、依赖扫描、策略验证、日志查询等）以插件形式集成到 Agent 的执行链路中。本文后续方法中所涉及的 MCP（Model Context Protocol）

可视为此类“工具接入与上下文交互”的一种工程化形态：通过统一的接口组织工具能力与返回结果，使得 Agent 能够在受控条件下进行检索、分析、生成与校验的闭环迭代。

在云原生安全场景下，AI Agent 的价值主要体现在：其一，面对漏洞公告、代码仓库与运维配置等多源异构证据，可借助检索与工具链实现更高效的上下文汇聚与复核；其二，将策略生成与验证过程结构化（如固定的 JSON 结构规范、配置片段语法检查、部署前置条件核对），可提高安全策略的可部署性与一致性；其三，通过人工在环（human-in-the-loop）与审计记录机制，使自动化生成与安全合规要求保持一致，提升最终交付的可控性。

2.3 国内外研究现状

围绕 Kubernetes 第三方应用的安全风险，国内外研究大体可分为“配置与清单风险分析”“最小权限与过度授权治理”“权限提升与运行时攻击场景建模/检测”三个方向。

（1）**配置与清单风险分析**。Kubernetes 的工作负载与权限往往以 YAML 清单形式交付，配置的可复制性也使得错误配置在开源生态中具有传播性。已有实证研究对开源 Kubernetes manifests 中的安全误配置进行了系统统计与归因，揭示了过宽权限、特权容器、弱隔离等问题的普遍性 [8]。该方向的价值在于刻画“风险在现实生态中的分布”，但其分析通常基于静态规则或启发式检查，难以充分覆盖不同集群环境与应用运行时差异所带来的动态性。

（2）**最小权限与过度授权治理**。最小权限原则是权限配置治理的基础思想 [5]。在实际系统中，冗余权限会扩大攻击面并提高误用概率 [6]；在 Kubernetes 场景下，第三方应用的过度权限被证明可被利用以实现集群关键资源的接管或扩大攻击影响 [7]。围绕“如何刻画应用的合理权限需求并识别过度授权”，已有工作从静态分析、配置推导与经验规则等角度展开探索（例如 EPSan[20]、KubeFence[21] 等），但在工程落地时仍面临权限需求随版本演进、环境差异导致的泛化困难，以及“可用性优先”带来的权限放宽等现实约束。

（3）**权限提升与运行时攻击场景建模/检测**。权限提升不仅来源于静态配置本身，也与 DevOps 流水线、运行时 API 调用与权限链条有关。既有研究总结了 Kubernetes 权限提升的典型路径与攻击链条，并给出可复现的攻击场景与对抗视角 [7, 9]。该方向强调对“攻击如何发生”的机制理解，但将研究



在上述研究基础上，课题组工作提出了面向权限风险的系统化检测框架 KubeGuard，覆盖过度权限、权限提升与敏感信息泄露三类风险，并分别提出最小权限集构建、规则化审计与敏感词监测等机制 [10]。该工作一方面，基于 NLP 技术识别应用的最小权限集来判定权限配置中是否存在过度授权；另一方面，采用动态的规则化审计机制检测多类权限提升风险；同时，通过监测 Pod 间消息传输并进行关键词匹配，对涉及敏感资源的内容触发告警，从而降低敏感信息泄露风险。进一步地，课题组实现了原型系统 KubeGuarder，并在开源 Kubernetes 应用与模拟场景上开展实验评估；结果表明，该框架在过度权限检测方面相较静态基线方法更为有效且更精确，并能够同时发现多类型的权限提升与敏感资源泄露风险 [10]。

在云原生场景下，漏洞补丁的发布与升级往往存在时间差；同时，生产环境的组件组合与权限边界具有显著异构性。对于权限提升类漏洞而言，这意味着处置流程通常需要在“补丁尚不可用或难以及时部署”的阶段，先采取**临时防控措施**以降低风险暴露面。基于这一现实需求，本文将动机聚焦于可动态下发的安全控制域，通过准入控制、网络隔离与运行时检测等手段，对攻击链中的关键环节施加约束，实现可快速部署、可验证、可回滚的风险缓解。

13

- **准入与策略约束：**基于 OPA/Gatekeeper 或 Kyverno 等准入策略，对特权容器、宿主机路径挂载、高风险 Linux capabilities、过宽 RBAC 绑定等高风险资源进行阻断或强制修正；
- **网络隔离与最小连通：**基于 NetworkPolicy 对命名空间与工作负载间通信进行白名单化，限制横向移动与数据外传路径；
- **运行时检测与响应：**基于 Falco 等运行时安全工具，监测异常系统调用、提权相关行为与逃逸迹象，并结合告警、隔离等处置流程降低影响扩散；
- **安全基线与最小权限：**结合 Pod 安全基线（如 Pod Security Standards）与账户/令牌治理，压缩默认配置下的权限暴露面。

面向权限提升漏洞的临时防控，自动化生成的方案需要同时满足三个要求：**可解释**（能够说明措施与风险点的对应关系）、**可执行**（给出可落地步骤，并明确验证与回滚边界）、**可复核**（关键结论可回溯到证据与环境假设）。然而，漏洞描述与运行环境信息往往存在噪声与缺失，不同控制手段对业务的副作用也存在差异，使得“生成—筛选—交付”难以依靠单一的启发式规则完成。为此，本文采用结构化产物、证据来源追踪与规则化质量评价相结合的方式，将策略生成过程约束到可检查的中间结果上，降低不确定性对交付质量的影响。

在上述约束下，引入大模型的核心动因在于提升对漏洞信息理解与策略生成的效率与覆盖度。面对公告、代码、讨论材料等多源非结构化证据，大模型可在检索增强上下文的支持下归纳漏洞成因，梳理权限边界与可控点，并生成多种候选的临时防控方案。与此同时，本文通过结构化字段设计、动作序列规范与门槛评价机制，将生成结果转化为可检查、可复核的交付产物，使其更契合 CVE 快速缓解与防控流程，并与必要的验证与审计环节保持一致。此外，本文在交付闭环中引入 human-in-the-loop（人工在环）机制：由安全工程师在门槛评价通过后对关键假设、证据引用、潜在业务副作用与回滚边界进行复核，并对安全策略部署决策作出最终确认。同时策略生成的每一步都可以人工检验中间结论与证据来源，并进行上下文操作和 prompt 优化等。

3 面向云原生权限提升漏洞的智能防控技术

本章给出本文方法体系和技术细节，包括形式化定义、总体流程和三个主要技术点。

3.1 形式化定义

3.1.1 基本对象

本节给出框架的形式化定义，用于精确刻画漏洞输入、策略生成与质量评价之间的关系。

定义 1 (漏洞实例): 令 $\mathcal{V}$ 表示漏洞实例集合，每个漏洞 $v \in \mathcal{V}$ 由三元组表示：

$$v = \langle \text{id}, d, m \rangle$$

其中 id 为 CVE 标识符， d 为 CVE 公告或其他文本描述， m 为元数据（如严重性、受影响组件等）。

定义 2 (安全上下文): 令 $\mathcal{C}$ 表示上下文集合，上下文 $c \in \mathcal{C}$ 是与漏洞 v 相关的证据性文本或结构化信息的集合。

在本框架中，上下文构建强调“证据来源可标注”，可将上下文表达为：

$$c = \{(t_i, \text{src}_i)\}_{i=1}^n$$

其中 t_i 为片段内容， src_i 为来源标识（例如文档路径、公告链接等）。

定义 3 (安全策略): 令 Π 表示安全策略空间，每个策略 $\pi \in \Pi$ 是一个结构化对象，包含三个核心字段：

$$\pi = \langle \text{layer}, \text{method}, \text{actions} \rangle$$

其中 layer 描述防控所在的技术层面（例如身份与权限、网络隔离、运行时检测等）， method 是防控方法的分类或摘要说明， actions 是具体可执行的防控操作序列，详见定义 3.1。

定义 3.1 (防控操作序列): 定义 actions 为一个有序的防控操作集合：

$$\text{actions} = [a_1, a_2, \dots, a_n]$$

其中每个操作 a_i 描述一个具体的防控步骤，可能涉及安全工具的使用或系统/集群配置的修改。为便于落地与复核，操作描述通常应包含：(1) 可执行的

步骤或命令；(2) 作用对象或组件（例如 NetworkPolicy、RBAC 配置等）；(3) 预期效果与验证方式；(4) 可能的副作用与回滚方案。

3.1.2 评价与选择

本节在统一符号与基本假设的前提下，给出候选策略质量的形式化表达，并定义两类评价尺度：**评分与排名**。评分刻画候选在各维度上的绝对水平，适用于同一候选集合内的比较与选择；但评分受评价主体偏好、证据充分性与业务约束影响，跨样本直接对比可能产生偏差。排名以相对次序刻画候选优劣，并显式结合候选集合规模，从而更适合在不同漏洞样本之间进行可比性分析。

定义 4（三维质量向量）：令 $\pi \in \Pi$ 为任一候选安全策略，定义其质量向量为

$$q(\pi) = (E(\pi), I(\pi), D(\pi)) \in [0, 1]^3$$

其中 E 为有效性（Effectiveness），指覆盖攻击面/阻断关键路径的程度， I 为无干扰性（Interference），意为对业务的副作用与误报风险越低越好， D 为可部署性（Deployability），评价某个安全策略中的执行步骤是否清晰、可操作且可验证。

定义 5（维度评分尺度）：由定义 4 可知，三维质量向量 $q(\pi)$ 的每一维均可视为从策略空间到区间 $[0, 1]$ 的分量函数。对任一维度 $\delta \in \{E, I, D\}$ ，定义维度评分函数

$$s_\delta : \Pi \rightarrow [0, 1]$$

并取 $s_E(\pi) = E(\pi)$ 、 $s_I(\pi) = I(\pi)$ 、 $s_D(\pi) = D(\pi)$ 。评分提供连续刻度，便于在同一候选集合内进行细粒度区分；但由于评价主体偏好、证据充分性与业务约束的差异，评分口径在跨样本或跨主体对比时可能不一致，从而影响数值尺度的可比性。

定义 6（维度排名尺度）：对给定漏洞实例 v 的候选集合 $\Pi_v \subseteq \Pi$ ，记候选总数 $n = |\Pi_v|$ 。对任一维度 $\delta \in \{E, I, D\}$ ，定义维度排名算子输出该维度上的相对次序：

$$R_\delta(\Pi_v) \rightarrow (\pi_{(1)}, \pi_{(2)}, \dots, \pi_{(n)})$$

其中 $\pi_{(1)}$ 表示维度 δ 上最优候选。概念上， $R_\delta(\Pi_v)$ 可由维度评分函数 s_δ 诱导：按 $s_\delta(\pi)$ 从大到小对 Π_v 排序即可得到上述序列。本文要求对每一维度给

出**严格排序**，不允许出现名次相同的情况；当出现评分相等时，仍需依据预先约定的确定性次级准则打破平分，以保证名次唯一。

等价地，可定义秩函数 $r_\delta : \Pi_v \rightarrow \{1, \dots, n\}$ 表示候选在该维度下的名次。引入秩函数的目的在于将相对次序转化为**可计算的数值对象**，便于后续进行多评审结果聚合、分歧度量与统计分析。在严格排序约束下， r_δ 在 Π_v 上给出一一对应的名次编码。

与评分相比，排名是一种**相对刻度**：它不依赖单一主观数值的绝对大小，而强调在同一候选集合内的对比结果；同时， n 的显式引入使不同漏洞样本的可达质量上限差异得以保留。对于某些 CVE，受制于现有安全工具能力与部署约束，其候选策略整体质量可能偏低，但排名仍可在该样本内部给出更稳健的选择依据。

3.2 总体框架

本文方法的总体流程由三个阶段组成，依次完成证据构建、策略生成与质量评价。整体输入为一个新披露的漏洞实例 v ；第一阶段将多源材料组织为可追溯的安全上下文 c ；第二阶段以 (v, c) 为输入，通过多智能体协同（安全知识智能体、漏洞分析智能体、策略生成智能体、策略评审与优化智能体）生成并迭代优化候选策略集合 Π_v ；第三阶段对 Π_v 进行三维质量评价，并基于评分与排名完成候选选择与排序，为后续部署与处置决策提供依据。

1. **开源应用漏洞安全知识库构建**：收集目标开源应用的漏洞数据与相关材料（如代码仓库、官方文档、安全公告与社区讨论），检索并汇聚与漏洞相关的证据，形成带来源标注的安全上下文 c 。
2. **基于多智能体协同的策略生成与优化**：以 v 与 c 为输入，通过多个专业智能体的协同工作——安全知识智能体提炼证据、漏洞分析智能体完成结构化分析、策略生成智能体输出候选策略、策略评审与优化智能体进行质量控制与迭代优化，最终得到候选集合 Π_v 。
3. **策略评估与质量控制**：对 Π_v 中的候选策略执行三维质量评估，基于评分与排名完成候选选择与排序。

3.3 开源应用漏洞安全知识库构建

3.3.1 漏洞收集与分类框架

本节阐述漏洞样本的收集流程与两级成因分类框架的构建方法。该分类框架面向策略生成场景：在输入侧提供相对稳定且可解释的风险语义线索，用以约束模型的推理路径，并辅助其对控制域与关键控制点进行选择，从而提升输出策略的可部署性、可验证性与副作用可控性。

(1) **云原生应用清单与仓库映射**。以 Kubernetes 及其生态开源应用为研究对象，首先从 CNCF Landscape 获取云原生应用清单，并对清单进行快照固化以保证可复现性。随后将条目名称映射到其官方代码仓库，形成后续漏洞关联与证据收集的目标集合。

(2) **漏洞收集与筛选**。在数据源选择上，综合使用 NIST NVD¹、MITRE CVE 官方库²与安全公告平台 GitHub Security Advisory³，检索 Kubernetes 及其生态应用中与权限提升相关的高风险漏洞。具体收集策略包括：

- **关键词约束**：在 CVE 描述与参考链接中匹配 Privilege Escalation、Authorization Bypass、Improper Access Control、RBAC、ServiceAccount、ClusterRole、Container Escape/Sandbox Escape/Breakout 等关键词；
- **严重性阈值**：采用 CVSS v3.x 评分，并以 ≥ 5.0 的高危与严重漏洞为主要对象；
- **人工复核**：对候选条目结合公告原文、厂商通告与社区讨论进行复核，剔除重复或与权限提升无关的记录，并补全攻击前提、影响面与可用缓解信息。

最终保留的结构化字段包括 CVE 编号、漏洞摘要、披露时间、受影响组件与版本范围、修复版本、CVSS 评分、参考链接，以及与修复相关的补丁提交记录等。

(3) **CWE 标注与分布统计**。为刻画漏洞成因层面的共性，本文对样本进行 CWE (Common Weakness Enumeration) 标注。CWE 是 MITRE 维护的软件弱点枚举体系，可用于将具体 CVE 归并到更抽象的弱点类型。本文依据公开映射与公告信息提取每个 CVE 的 CWE 标签，并统计不同 CWE 的出现频次。

¹NIST NVD (National Vulnerability Database): <https://nvd.nist.gov/>

²MITRE CVE: <https://cve.mitre.org/>

³GitHub Security Advisory: <https://github.com/advisories>

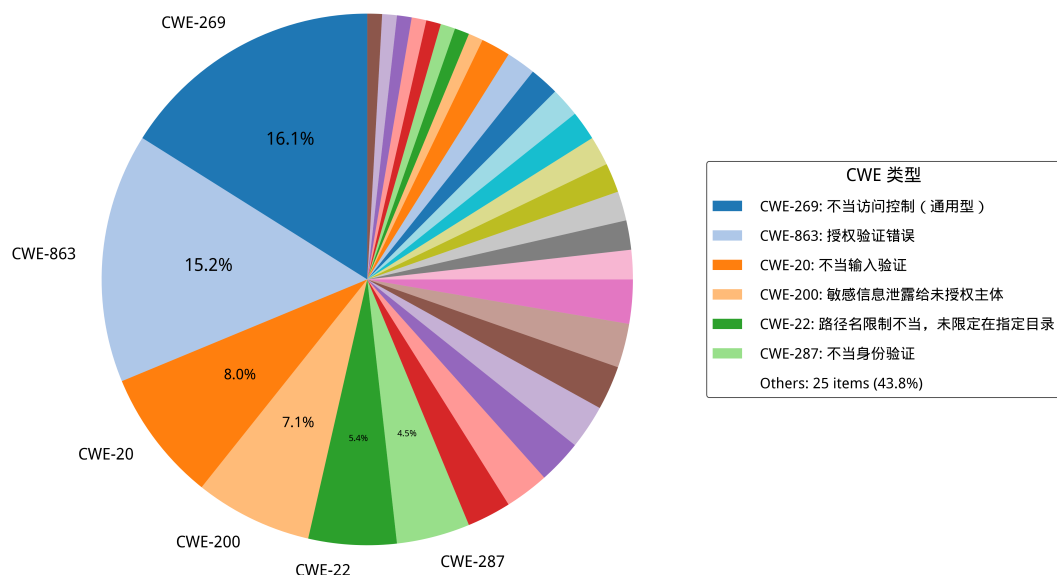


图 3-1: Kubernetes 权限提升 CVE 对应的 CWE 类型占比

CWE 体系在组织形态上并非严格正交的分类结构：同一漏洞可能对应多个弱点条目，且不同条目在抽象层次上跨越具体机制与高层概念，使得类别之间存在交叉与重叠。在本文收集的 55 个权限提升漏洞中，36% 的 CVE (20 个样本) 同时关联两个及以上 CWE 标签。为客观反映 CWE 分布，统计时对每个 CVE 赋予单位权重 1.0，多标签情况下各 CWE 均分该权重。统计结果显示，访问控制类弱点（如 CWE-863 授权不当、CWE-269 权限管理不当）与输入处理类弱点（如 CWE-20 输入验证不当）在多个样本中共现，反映出漏洞成因的复合特征。上述非正交特性导致单一 CWE 标签难以为防控策略选择提供唯一且稳定的语义约束，因此需构建面向防控动作的成因分类框架。

(4) 面向防控的成因分类框架。为解决“根因标签不稳定/不互斥”对策略生成的影响，本文提出面向云原生权限提升漏洞的两级成因分类框架，并将每个漏洞映射为（一级类，二级类）标签：

- **一级分类**刻画成因大类（如安全逻辑缺陷、配置错误、代码注入、敏感信息泄露等），用于提供高层风险语义与控制域提示；
- **二级分类**进一步细化为更贴近防控动作选择的类型（例如授权验证不完善、默认不安全配置等），用于将漏洞分析结果对齐到可操作的控制点（RBAC 最小权限、准入控制、网络隔离、运行时策略与审计等）。

在策略生成阶段，分类结果作为提示词中的结构化约束，有助于模型将“漏

洞语义 → 控制域 → 可执行动作序列”对齐，减少泛化建议与无关信息，提高策略的可部署性与针对性。具体分类定义如下：

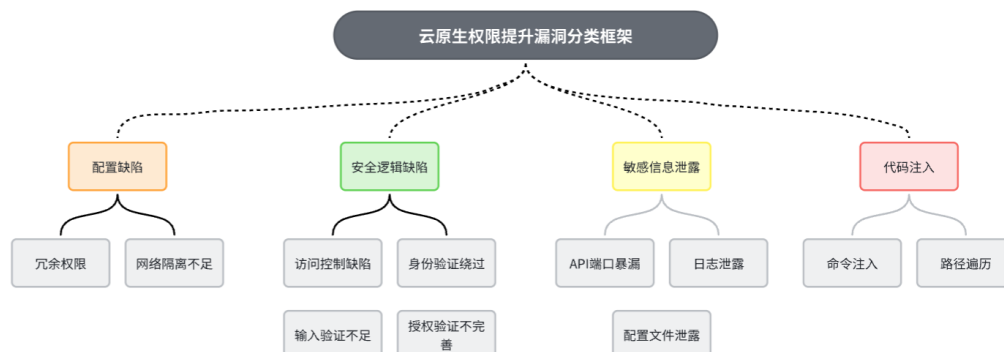


图 3-2: Kubernetes 容器化应用权限提升攻击方式分类

1. 配置缺陷：应用或基础设施的配置不当导致的安全漏洞，包括：

- 过度权限配置：Kubernetes RBAC 配置不当、Role/ClusterRole 规则范围过宽或绑定关系不合理等，使低权限主体获得超出其职责的资源访问与高危操作能力。该类问题在第三方应用接入场景中尤为常见，并已被证明可被用于接管集群关键资源 [6, 7]。
- 网络隔离不足：容器、Pod 或服务间缺乏网络隔离，导致攻击者可以横向移动获取其他权限上下文。

2. 安全逻辑缺陷：应用代码中的安全机制设计或实现不当，包括：

- 访问控制缺陷：应用的权限检查不完全或存在逻辑漏洞，允许绕过权限验证；
- 身份验证绕过：认证机制设计缺陷，攻击者可以冒充他人身份或跳过认证步骤；
- 授权验证不完善：权限检查在某些操作路径或边界条件下缺失。

3. 敏感信息泄露：重要的身份、密钥或权限信息意外暴露，包括：

- API 端口暴露：管理接口、内部 API 或调试端口未受保护暴露在网络上；
- 日志泄露：日志文件包含敏感信息（如 token、密钥、命令历史）；
- 配置文件泄露：配置文件包含硬编码的凭证或密钥。

4. 代码注入：应用对用户输入的处理不当，允许注入恶意代码，包括：

- 命令注入：应用直接执行用户提供或可控的命令，攻击者可以执行任意系统命令；
- 路径遍历：应用文件访问检查不完整，攻击者可以访问系统其他部分的

文件。

3.3.2 知识库构建

知识库构建阶段的目标是将与漏洞 v 相关的多源材料组织为带来源标注的证据集合 $c = \{(t_i, \text{src}_i)\}_{i=1}^n$ ，并以嵌入式模型构建一个可检索的向量索引，以支持后续“按需取证”的上下文供给。

(1) 材料收集与最小化预处理。对每个漏洞关联的开源应用仓库，收集代码、配置与文档等可读文本；同时纳入与漏洞相关的安全公告、Issue/PR 讨论与修复提交信息。对文本进行最小化规范化（统一换行与空白），并过滤超长或二进制内容，以降低噪声与资源开销。

(2) 结构化分块。为兼顾证据粒度与可复核性，本文对不同材料采用差异化分块：对代码/配置按行切分并尽量对齐函数或语义边界；对文档按段落与窗口切分并保留适当重叠，以减少跨块语义断裂。

(3) 向量化与索引。对每个文本块使用嵌入式模型生成向量表示。本文选用 `sentence-transformers/all-mpnet-base-v2` 作为编码器，将文本块映射到统一的向量空间。基于向量表示，采用 FAISS 构建近邻索引以支持相似度检索，从而为后续按需检索证据提供高效的底层能力。为保证可追溯与可复现，索引与元数据进行同步持久化，内容包括向量索引文件、块级元数据、以及构建配置。块级元数据记录来源路径或链接、块 ID、块类型与范围等信息；构建配置记录分块参数与检索阈值等关键设置。

上述流程得到的向量知识库为后续策略生成提供了“可追溯证据”的供给能力：模型在推导控制措施时可以检索到与漏洞相关的关键事实与上下文片段，并通过来源标识完成快速定位与复核。

3.4 基于多智能体协同的策略生成与优化

策略生成阶段以漏洞实例 v 与安全上下文 c 为输入，生成候选策略集合 Π_v （图3-3）。流程由四类智能体分工完成：安全知识智能体整理证据；漏洞分析智能体给出结构化分析；策略生成智能体产出候选；策略评审与优化智能体基于评审反馈修订，以降低误报与运维噪声并提升可部署性。

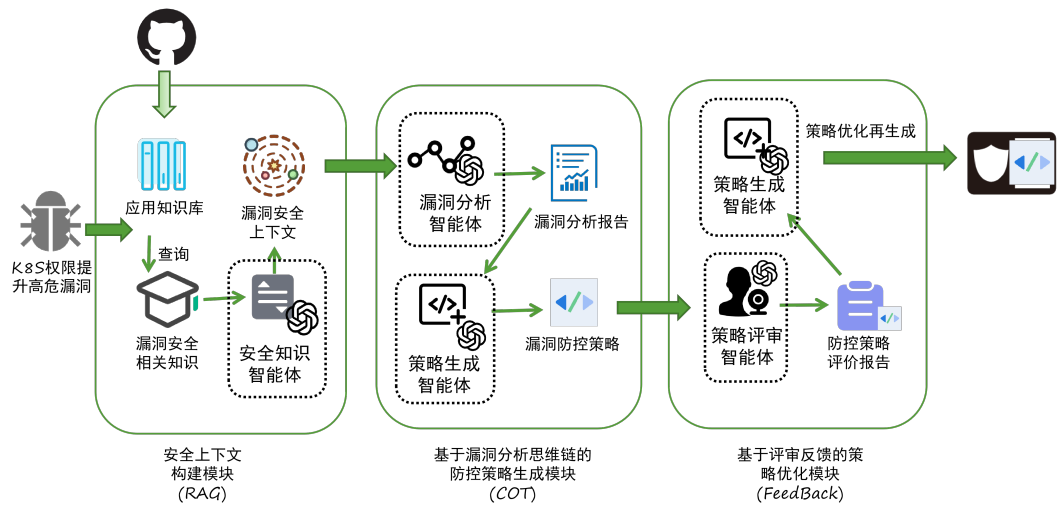


图 3-3: 多智能体协同的安全策略生成框架

3.4.1 安全上下文构建

安全上下文构建以“证据可追溯”为目标,形成证据集合 $c = \{(t_i, \text{src}_i)\}_{i=1}^n$ 。知识库阶段为每个片段记录来源元数据（路径/URL/版本/位置）以便回溯；生成阶段从知识库与安全材料检索候选证据，并由安全知识智能体去重与提炼，输出触发条件、受影响组件、权限边界与可控点，同时保留对应的 src_i 供复核。

安全知识智能体 PROMPT 模板

你是云原生安全专家，负责生成符合结构规范的安全策略 JSON 数据，
↪ 熟悉 Kubernetes 漏洞与缓解措施。

输入数据

<VULN_SUMMARY>

任务目标

基于提供的漏洞信息和相关知识，生成一个完整的JSON对象，
↪ 严格遵循下面给出的结构规范。

工具提示

可选用 OPA、Kyverno、Falco、NetworkPolicy 等工具，并在
↪ mitigation 字段写明工具与作用。

输出规则

1. 仅返回 JSON: 只输出单个 JSON 对象, 不要附加说明、Markdown、
→ 表格或其它文本。
2. 严格格式: 字段、层级、顺序必须与上方给出的结构规范/示例一致,
→ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失: 无法确定时写 "(info needed)", 并在字段或
→ assumptions 中说明假设。
4. 配置转义: YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解: 禁止仅写“升级补丁”,
→ 需给出可执行的控制措施与配置步骤(如适用)。
6. 一致性: 内容必须与输入/分析/评审结论一致, 禁止引入无依据内容。
→

3.4.2 基于漏洞分析思维链的策略生成

该步骤在结构化约束下完成“漏洞分析—技术选型—策略生成”, 并输出关键中间结论, 便于复核。与仅输出自然语言建议不同, 本文统一约束中间产物与最终策略为固定的 JSON 结构, 以减少信息缺失与表述歧义。

首先, **漏洞分析智能体**基于漏洞摘要与安全上下文 c 输出结构化分析结果, 覆盖漏洞类型、攻击向量、影响范围与风险等级, 并给出与本文分类体系一致的风险类别。随后, **策略生成智能体**在提示词模板与思维链 (CoT) 引导下, 以分析结论为约束完成控制手段选择与动作序列生成: 结合平台可用能力在身份与权限、网络隔离、运行时安全与策略执行等控制域中确定优先级与实施路径, 并将其映射为最终策略 JSON; 当信息不足以确定时, 以 (info needed) 显式标注缺失信息并给出必要假设。

漏洞分析智能体思维链 PROMPT 模板

你是 Kubernetes 安全分析师, 请对以下 CVE 进行分析并分类。

输入信息

<VULN_SUMMARY>

相关知识上下文

<RAG_CONTEXT>

分析要求

请从以下维度分析 CVE:

1. 漏洞类型: 详细说明这是什么类型的漏洞
2. 攻击向量: 漏洞如何被利用
3. 影响范围: 漏洞可能造成的影响
4. 风险等级: 基于 CVSS 评分和实际影响评估
5. 漏洞分类: 从以下分类中选择最合适的一个:
 - 内部权限失控
 - 过度权限配置
 - 代码注入
 - 敏感信息泄露
 - 缺乏网络隔离
 - Kubernetes 漏洞
 - 容器运行时漏洞

输出格式

请返回 JSON 格式的分析结果:

```
{  
  "vulnerability_type": " 漏洞类型描述",  
  "attack_vector": " 攻击向量描述",  
  "impact_scope": " 影响范围描述",  
  "risk_level": " 高/中/低",  
  "category": " 选择上述分类之一",  
  "key_findings": " 关键发现总结"  
}
```

输出规则

1. 仅返回 JSON: 只输出单个 JSON 对象, 不要附加说明、Markdown、
↪ 表格或其它文本。
2. 严格格式: 字段、层级、顺序必须与上方给出的结构规范/示例一致,
↪ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失: 无法确定时写 "(info needed)", 并在字段或
↪ assumptions 中说明假设。

4. 配置转义：YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解：禁止仅写“升级补丁”，
→ 需给出可执行的控制措施与配置步骤（如适用）。
6. 一致性：内容必须与输入/分析/评审结论一致，禁止引入无依据内容。
→

策略生成智能体 PROMPT 模板

基于 CVE 分析和防控技术选型，生成完整的安全策略。

CVE 分析

<ANALYSIS_RESULT>

防控技术选型

<MITIGATION_ANALYSIS>

相关知识上下文

<RAG_CONTEXT>

策略生成要求

你必须返回一个有效的 JSON 对象，格式如下：

```
{
  "cve_id": "string - CVE 编号",
  "description": "string - 中文漏洞简介，
    → 聚焦攻击面和触发条件",
  "application": "string - 受影响的业务组件和运行环境假设",
  "threat_analysis": {
    "preconditions": "string - 漏洞触发的前提条件",
    "attack_path": "string - 攻击链描述",
    "blast_radius": "string - 潜在影响范围"
  },
}
```

```

"risk_category": "string - 必须是以下之一：
↳ 内部权限失控|过度权限配置|代码注入|敏感信息泄露|缺乏网
↳ 络隔离|Kubernetes 漏洞|容器运行时漏洞",
"mitigation": {
  "layer": "string - 安全层级：
↳ 身份与权限|网络|运行时|策略执行",
  "method": "string - 具体方法，如：设置Falco规则|应用
↳ NetworkPolicy|配置 Kyverno|部署 OPA Gatekeeper",
  "action": "string - 详细执行步骤，包含 YAML 配置片段"
}
}

```

工具提示

你可以结合分析和选型结果，灵活选择并说明如 OPA、Kyverno、Falco、

- ↳ NetworkPolicy 等主流安全工具。建议在 mitigation
- ↳ 字段中明确工具名称及其作用，但不强制定限。

输出规则

1. 仅返回 JSON：只输出单个 JSON 对象，不要附加说明、Markdown、
 - ↳ 表格或其它文本。
2. 严格格式：字段、层级、顺序必须与上方 schema/示例一致，
 - ↳ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失：无法确定时写 "(info needed)"，并在字段或
 - ↳ assumptions 中说明假设。
4. 配置转义：YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解：禁止仅写“升级补丁”，
 - ↳ 需给出可执行的控制措施与配置步骤（如适用）。
6. 一致性：内容必须与输入/分析/评审结论一致，禁止引入无依据内容。
 - ↳

为保证与本文形式化定义一致，策略内容采用 layer/method/actions 的表达方式：其中 layer 与 method 描述防控所在安全域与方法类别；actions

以可执行步骤序列承载具体控制措施。在工程化输出中，`actions` 可等价展开为策略 JSON 的 `mitigation.action` 字段，采用“1.. 2..”的步骤格式，并包含验证要点与回滚边界；当需要给出 YAML/CLI 片段时，统一使用转义换行 `\n` 以保持 JSON 合法性。

- **layer 字段：**防控所在的技术层面，用于分类防控策略所涉及的安全域，包括但不限于：
 - 身份与权限：RBAC、认证强化、ServiceAccount 管理等；
 - 网络隔离：防火墙规则、NetworkPolicy、流量隔离等；
 - 运行时检测：容器监控、系统调用检测、入侵检测等；
 - 审计与日志：日志记录、事件审计、合规监控等。
- **method 字段：**防控方法的摘要或分类，用于快速理解防控策略的整体思路，例如：
 - 网络隔离与认证强化；
 - 权限最小化与配置加固；
 - 检测与阻断相结合。
- **actions 字段：**具体可执行的操作序列，遵循定义 3.1（见定义 3.1.1），包含：
 - 对应的安全工具或修改对象（如 Cisco DNA Center、Kubernetes RBAC 配置）；
 - 具体的执行步骤，包括命令、参数、验证方式；
 - 副作用评估与回滚方案。

为提高交付可用性，本文对候选策略施加基本质量要求：操作序列应可在短时间内启动；至少覆盖一个明确的防控层次；包含可验证的执行步骤；对潜在副作用与回滚边界作出清晰说明。

候选策略集合 Π_v 的生成可采用多策略并行方法：对同一漏洞产生多个结构化策略对象，以探索不同的防控层次和防控方法。由于所有候选共享统一的结构表示（`layer`、`method`、`actions`），它们可直接进入后续的评价与排名流程。

在交付前，生成结果需进行规范化处理：统一各字段的格式、补齐缺失的操作步骤、清理冗余信息，并在数据缺失或不完整处显式标注假设条件和依赖的前置条件。

3.4.3 基于评审反馈的策略优化

该步骤通过“评审—反馈—再生成”对候选策略进行质量控制，降低误报与运维噪声，提升可部署性与一致性。系统并行生成多份候选形成 Π_v ，由**策略评审智能体**给出结构化评审（有效点、风险、验证要点与缺失信息），再由**策略优化智能体**合并可行措施、剔除不可行建议，并补齐前置条件与回滚方案。

策略评审智能体提示词模板

你是一名资深 Kubernetes 安全架构师，负责评审多份安全策略。

- ↪ 目标是识别关键控制措施，定位误报与运维噪声来源，
- ↪ 并明确上线前的验证需求。

输入说明

- CVE 信息：含编号、摘要、CVSS、受影响场景等
- 原始策略列表：包含 profile 标签、JSON 策略、生成摘要
- 补充约束：可选背景、现有控制、风险偏好

评审要求

1. 有效性 (effective)：提炼每份策略最具价值、可快速落地的控制点
2. 干扰/误报 (noise)：指出可能带来误报或运维噪声的配置，
 - ↪ 并提出缓解思路
3. 验证 (validation)：明确上线前或运行期需要的验证步骤、
 - ↪ 前置条件或监控指标
4. 一致性：若发现明显错误、冲突、缺失信息，必须指出
5. 输出精炼：不要打分，避免冗长段落

输出格式

仅返回 JSON，结构如下：

```
{
  "overall_findings": [" 整体发现 1"],
  "per_strategy": [
    {
      "profile": "llm_profile_name",
```

```

        "strengths": [" 有效点 1"],
        "gaps": [" 缺口 1"],
        "risks": [" 潜在风险 1"],
        "noise_sources": [" 潜在噪声来源 1"],
        "validation_needs": [" 验证要点 1"],
        "actionable_feedback": [" 落地改进建议 1"]
    }
],
    "missing_information": [" 待补充信息 1"],
    "assumptions": [" 关键假设 1"],
    "confidence": " 高/中/低"
}

## 输出规则
1. 仅返回 JSON: 只输出单个 JSON 对象, 不要附加说明、Markdown、
   ↳ 表格或其它文本。
2. 严格格式: 字段、层级、顺序必须与上方给出的结构规范/示例一致,
   ↳ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失: 无法确定时写 "(info needed)", 并在字段或
   ↳ assumptions 中说明假设。
4. 配置转义: YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解: 禁止仅写“升级补丁”,
   ↳ 需给出可执行的控制措施与配置步骤(如适用)。
6. 一致性: 内容必须与输入/分析/评审结论一致, 禁止引入无依据内容。
   ↳

```

策略优化智能体提示词模板

你是一名 Kubernetes 安全架构师,

↳ 负责在评审结果的基础上生成最终的优化策略。

输入说明

- CVE 信息: 含编号、受影响系统、漏洞摘要
- 原始策略列表: 来自不同 LLM 的安全策略
- 评审结论: 上一阶段输出的评审 JSON

- 补充约束：可选背景、现有控制、风险偏好

任务

1. 汇总评审中确认的有效控制，剔除不可行或噪声过高的措施
2. 生成符合既定结构规范的最终策略，`mitigation.action` 使用 1.
 - ↪ ... \n2. ... 结构并保持与历史输出一致，
 - ↪ 尽量包含可直接落地的策略代码片段。
3. 若信息缺失，写入 `assumptions` 或在相关字段标注 (`info needed`)

输出格式

仅返回 JSON，结构如下：

```
{
  "optimized_strategy": {
    "cve_id": "CVE 编号",
    "description": " 中文描述（无换行）",
    "application": " 受影响环境或（信息缺失）",
    "threat_analysis": {
      "preconditions": "...",
      "attack_path": "...",
      "blast_radius": "..."
    },
    "risk_category": " 风险分类",
    "mitigation": {
      "layer": " 身份与权限 / 策略执行",
      "method": " 与示例保持一致的组合描述",
      "action": "1. ... \n2. ... \n3. ..."
    }
  },
  "improvements": [
    {
      "topic": " 改进主题",
      "details": " 简述优化内容及预期效果",

```

```

        "derived_from": ["profile_a"]
    }
]
}

```

输出规则

1. 仅返回 JSON: 只输出单个 JSON 对象, 不要附加说明、Markdown、
→ 表格或其它文本。
2. 严格格式: 字段、层级、顺序必须与上方给出的结构规范/示例一致,
→ 所有字符串使用双引号并保持合法 JSON。
3. 信息缺失: 无法确定时写 "(info needed)", 并在字段或
→ assumptions 中说明假设。
4. 配置转义: YAML/CLI 片段的换行统一写成 `\\n`。
5. 具象缓解: 禁止仅写“升级补丁”,
→ 需给出可执行的控制措施与配置步骤(如适用)。
6. 一致性: 内容必须与输入/分析/评审结论一致, 禁止引入无依据内容。
→

此外, 策略生成智能体集成 MCP, 对生成的 YAML/CLI 等片段做语法检查, 将可部署性相关的低级错误前移。检查重点包括: **语法正确性** (如 YAML 缩进、引号闭合)、**最小格式约束** (必要字段、占位符残留)、**结构一致性** (与 layer/method/actions 的对应关系)。

当检测到语法错误或不满足最低格式约束时, MCP 将返回结构化错误信息 (错误类型、定位位置、最小修复建议等)。策略生成智能体把该错误信息作为反馈输入, 触发对相关片段的**定向迭代修正**: 仅修改出错片段及其必要上下文, 保持其他已通过检查的内容不被无关改写; 修正后再次调用 MCP 复检。该迭代过程设置最大轮次上限, 以避免无限循环; 若达到迭代上限仍无法通过检查, 则将该候选标记为需人工复核或需补充信息, 并在后续质量评价环节中置末位, 以保证最终交付策略具备基本可部署性。

3.5 策略评估与质量控制

该阶段对候选策略集合进行结构化评价，输出三维质量向量 $q(\pi)$ 及其在各维度上的排名结果，整体流程遵循结构化输入 → 规则化校验 → 证据支撑的结论的路径，以提高结论的可验证性与可复核性。

3.5.1 评价算子

为保证评价结果的可复核性与可比性，本文将评价算子划分为两类：**LLM 支持的评价算子**与**专家人工评价算子**。两类算子在评价维度与输出形式上保持一致，均以三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ 为依据，输出按维度的排名结果；差异在于评价主体与证据使用方式：前者由评价智能体在提示词约束与证据上下文支持下给出结构化结论，后者由安全专家结合现场约束与经验进行复核与调整。

(1) LLM 支持的评价算子。该算子以候选策略集合 Π_c 为输入，在固定的评价准则与输出结构规范约束下，对候选策略在有效性 E 、无干扰性 I 、可部署性 D 三个维度进行比较并给出排序。为降低模型的提示偏置与先验印象影响，评价前对候选策略进行**随机化与匿名化处理**：

- **随机化**：对候选策略列表进行随机打乱，避免顺序效应；
- **匿名化**：移除或遮蔽与生成来源相关的元信息（如模型名称、profile 标签、生成时间等），仅保留内容本身，并为每条候选分配不可语义化的匿名编号（如 $S_1, S_2, \dots$ ）。

为保持可追溯性，系统同时记录匿名编号与原始候选的映射表，供最终复核与实验统计使用。

(2) 专家人工评价算子。该算子由安全专家基于同一评价维度与检查清单，对候选策略进行复核：重点验证关键前置条件是否成立、控制点是否覆盖主要攻击路径、策略副作用与回滚边界是否明确，以及配置片段是否符合组织内基线与现场约束。专家评价可以对 LLM 排名结果进行解释性标注与必要校正，并在存在高风险不确定性时给出需补充信息或需灰度验证的结论。

评价智能体 PROMPT 模板（LLM 支持的评价算子）

```
EVALUATION_PROMPT_TEMPLATE = r"""# Kubernetes
↪ 策略三维评价与排名
```


你是一名资深云原生安全评审专家，

↪ 负责对候选安全策略进行三维评价与排名。

输入说明

- 候选策略列表（已匿名化、已随机化）：每项包含 `id` 与

↪ `strategy_json`

- 评价维度：有效性 `E` / 无干扰性 `I` / 可部署性 `D`

评价要求

1. 仅比较内容本身：不得根据任何生成来源、模型、提示词、

↪ 作者等信息推断质量（输入中已匿名化）。

2. 三维分别排序：分别给出 `E/I/D` 三个维度的从优到劣的排名列表。

3. 维度解释口径：

- `E`（有效性）：是否覆盖关键攻击路径/控制点，

↪ 是否具备明确的风险降低机制；

- `I`（无干扰性）：对业务影响、误报/运维噪声、

↪ 过度限制风险越低越好；

- `D`（可部署性）：步骤清晰、可操作、可验证、可回滚，

↪ 且配置片段语法/结构合理。

4. 排序约束：不允许出现名次相同的情况；当难以区分时，

↪ 仍需依据明确的次级准则给出严格排序，并在 `notes`

↪ 中简要说明判别依据。

输出格式

仅返回 `JSON`，结构如下：

```
{
  "rankings": {
    "E": ["S3", "S1", "S2"],
    "I": ["S2", "S1", "S3"],
    "D": ["S1", "S3", "S2"]
  },
```

```

    "notes": ["如发现明显语法/结构问题，
    ↪ 或在必须打破接近同分的情况下使用了次级判别准则，
    ↪ 可在此简要说明。"]
}

## 输出规则
1. 仅返回 JSON：不要附加说明、Markdown、表格或其它文本。
2. 不得泄露匿名映射：不得尝试恢复或猜测候选来源。
"""

```

3.5.2 多智能体评估体系

依据第 3.1 节的形式化定义，本文将候选策略 $\pi \in \Pi$ 的质量表示为三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，并在每个维度 $\delta \in \{E, I, D\}$ 上要求对给定候选集合 Π_v 输出严格排序 $R_\delta(\Pi_v)$ （定义 4-6）。因此，多智能体评估体系的目标可表述为：对同一候选集合 Π_v ，引入多个评价主体 $\mathcal{H} = \{h_1, \dots, h_m\}$ ，由每个 $h \in \mathcal{H}$ 独立实现维度排名算子 $R_\delta^{(h)}(\Pi_v)$ ，并输出三维严格排序 $\{R_E^{(h)}, R_I^{(h)}, R_D^{(h)}\}$ ，作为后续聚合与选择的基础信号。

为降低单一模型偏差与不稳定性对结论的影响，本文采用多评审的同口径可审计组织方式：由多个不同来源或不同配置的基于大型语言模型（LLM）的评价算子对同一输入进行**独立评审**，再将各评审结果聚合为总体排序。该设计遵循两条原则：其一，**多样性**，通过引入不同能力侧重的评审主体覆盖更多评价视角（例如对有效性证据链的敏感度、对业务干扰的谨慎程度、对可部署细节的严格性）；其二，**独立性**，各评价算子在同一评价准则与输出结构规范约束下独立输出，避免评审主体之间接触对方结论而引入一致性偏置。

具体实现上，系统对通过规则化校验的候选集合 Π_v 执行匿名化与随机化处理后，将相同输入分发给各 $h \in \mathcal{H}$ 。在输出层面，本文以定义 6 的排名尺度为主：每个 h 仅需在每个维度上给出一个全序排序，并可在必要时返回简短的判别依据与风险提示。该设计避免了评分尺度（定义 5）在不同评审主体之间的绝对口径差异，从而更利于跨样本复现与统计分析。为增强鲁棒性与可复核性，多智能体评估体系引入如下控制机制：

- **一致性约束**：评价输出必须严格遵循既定的结构规范（仅输出三维严格

排序与简短备注)，并显式禁止使用候选来源信息推断质量（输入已匿名化）。

- **评审权重（等权）：**本文不区分不同评审主体（不同 LLM 或专家复核）的可靠性差异，默认采用等权设置：对 $m = |\mathcal{H}|$ 个评审主体，取 $w_h = \frac{1}{m}$ ，并满足 $w_h \geq 0$ 且 $\sum_{h \in \mathcal{H}} w_h = 1$ 。该设置更简洁、可复现，也避免引入额外的权重估计过程与主观性。
- **分歧触发复核：**当不同评审在某策略/某维度上分歧显著时，将该策略标记为高不确定性，优先进入专家复核或补充信息流程，避免仅凭单一评审结论做交付决策。

在上述体系下，评价算子的输出不再是不可解释的单点分数，而是可追踪的多源排序信号；下一小节将给出将多智能体三维排序聚合为总体质量评价与总排名的数学模型。

3.5.3 多源排名聚合模型

本节给出一个面向实验可复现的聚合模型。其核心目的不是引入新的主观打分，而是在多评审等权的前提下形成可审计的共识排序。具体而言，在每个维度 $\delta \in \{E, I, D\}$ 上，评价算子仅输出严格排序；系统将排序映射为秩并进行加权聚合。三维维度权重 $\alpha_E, \alpha_I, \alpha_D$ 反映业务场景偏好，可按需求进行定制化设定；评审权重则采用等权设置 $w_h = \frac{1}{m}$ ($m = |\mathcal{H}|$)，以保证模型简单且易复现。

(1) 输入：多评审排序结果。令 Π_v 表示通过规则化校验的候选集合， $|\Pi_v| = n$ ；令 $\mathcal{H} = \{h_1, \dots, h_m\}$ 表示参与评审的评价算子集合（可为多个评审 LLM，必要时可包含专家复核）。每个 $h \in \mathcal{H}$ 在维度 $\delta \in \{E, I, D\}$ 上输出一个严格排序：

$$R_\delta^{(h)}(\Pi_v) \rightarrow (\pi_{(1)}^{\delta, h}, \pi_{(2)}^{\delta, h}, \dots, \pi_{(n)}^{\delta, h})$$

其中 $\pi_{(1)}^{\delta, h}$ 表示在评审 h 的维度 δ 上最优候选。排序在匿名编号（S1、S2、...）上进行，最终通过映射表恢复到原策略用于审计与交付。

(2) 将排序映射为秩函数。为便于计算，定义秩函数 $r_\delta^{(h)} : \Pi_v \rightarrow \{1, \dots, n\}$ ：

$$r_\delta^{(h)}(\pi) = 1 + |\{\pi' \in \Pi_v \mid \pi' \succ_\delta^{(h)} \pi\}|$$

其中 $\pi' \succ_\delta^{(h)} \pi$ 表示在 h 的维度 δ 排序中 π' 严格优于 π 。在严格排序约束下，每个候选获得唯一秩，且 $r_\delta^{(h)}$ 为一一映射。

上述映射不引入新的偏好假设：它仅将排序的“相对位置”编码为整数，从而使排序结果可以直接进入后续的加权 Borda 聚合与分歧度量计算。

(3) 按维度聚合。本文采用基于秩的加权 Borda 聚合和等权设置聚合评分。对 $m = |\mathcal{H}|$ 个评审主体取 $w_h = \frac{1}{m}$ ，并满足 $w_h \geq 0$ 且 $\sum_{h \in \mathcal{H}} w_h = 1$ 。对维度 δ 定义该维度的聚合得分：

$$S_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot (n - r_\delta^{(h)}(\pi))$$

并将其归一化为 $\hat{S}_\delta(\pi) \in [0, 1]$ ：

$$\hat{S}_\delta(\pi) = \frac{S_\delta(\pi)}{n-1}$$

于是可得到维度 δ 的**共识排序** $\bar{R}_\delta$ ：按 $\hat{S}_\delta(\pi)$ 从大到小进行排序。为保证输出为严格排序，若出现数值相等，则按预先约定的确定性规则打破平分，例如依次比较 $\bar{r}_\delta(\pi)$ （越小越优）并以匿名编号作为最终判别。

为刻画“评审一致性/不确定性”，可进一步定义分歧度量，例如秩的加权方差：

$$U_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot (r_\delta^{(h)}(\pi) - \bar{r}_\delta(\pi))^2, \quad \bar{r}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot r_\delta^{(h)}(\pi)$$

当 $U_\delta(\pi)$ 较大时，表示该策略在该维度上存在明显分歧，需在后续决策中优先人工复核。

(4) 总体质量评价：三维共识到总排名。在获得三维共识质量后，给出总体质量函数 $Q : \Pi_v \rightarrow [0, 1]$ ：

$$Q(\pi) = \alpha_E \hat{S}_E(\pi) + \alpha_I \hat{S}_I(\pi) + \alpha_D \hat{S}_D(\pi), \quad \alpha_E + \alpha_I + \alpha_D = 1$$

其中 $\alpha_E, \alpha_I, \alpha_D$ 是三维度的**场景权重**，用于将“有效性/无干扰性/可部署性”的偏好映射到最终排序。该权重可按业务场景配置：例如在**应急与临时缓解**场景下可提高 α_E, α_D ；在**生产稳定性优先**场景下可提高 α_I 。

在本文的安全防控语境下，若无额外业务约束，默认偏好应体现“**有效性优先**”：即 $\alpha_E > \alpha_I > \alpha_D$ 。例如可取 $\alpha_E = 0.5, \alpha_I = 0.3, \alpha_D = 0.2$ 作为默认基线（具体数值可结合组织的风险偏好与运维能力进行微调）。

为避免“权重选择导致结论偶然变化”，实验中可对 α 做简单敏感性分析（例如在合理范围内扰动 α 并观察总排名是否稳定），将不稳定样本优先交由专家复核。最终**总排名** $\bar{R}$ 由 $Q(\pi)$ 从大到小排序得到；为保证输出为严格排序，若出现 $Q(\pi)$ 相等，则按 $(\hat{S}_E, \hat{S}_D, \hat{S}_I)$ 的词典序进行细化，并在仍无法区分时以匿名编号作为最终判别，从而得到可审计的最终交付结论。

4 面向云原生权限提升智能化漏洞防控系统工具

4.1 需求分析

云原生场景下的权限提升（Kernel/Container Privilege Escalation, KPE）漏洞证据分散、修复策略复杂，且往往需要在安全性与业务影响之间权衡。基于此，本系统原型工具按 **human-in-the-loop**（人在回路）范式设计：模型负责生成候选与给出建议，人类负责证据核验、策略修订与最终决策，并通过结构化留痕保证过程可追溯、结果可复现。整体流程以“证据汇聚—策略生成/优化—质量评价与审阅—发布与回滚”为闭环。

目标与范围

本系统聚焦于 KPE 漏洞的策略生成与质量评估：从证据获取、生成与优化，到审阅决策与留痕回滚。原型阶段不直接对接真实集群执行，而将“发布”定义为输出可落地策略与采纳决策，并将证据、产物与决策过程持久化，支撑复现与后续工程化。

用户角色与使用场景

系统面向以下典型角色：

- **安全工程师/平台运维（SRE）**：浏览漏洞证据与候选策略，评估有效性与风险，给出采纳或带监控采纳的决策，并对策略做必要的人工修订。
- **评审/标注人员**：对漏洞或策略进行三维评分并给出评语，形成可审计的人工标注数据，用于后续分析或学习排序偏好。
- **研究/算法工程师**：对多方法/多模型输出进行对比，查看统计仪表盘与样例钻取结果，分析不同方法在三维指标上的表现差异。

功能性需求

结合原型实现，本系统的核心功能需求如下：

1. **证据与知识汇聚**：支持加载漏洞清单与 CVE 原文（Markdown），展示第三方应用与漏洞摘要等结构化字段，并支持多模型/多方法结果的聚合浏览。

2. **交互式策略生成与优化**：提供端到端管线会话（session）管理，支持分阶段执行检索、规划、推理、策略生成、评审与优化；支持人工在环对中间产物进行编辑回写。
3. **三维质量评价与审阅**：对候选策略进行“有效性”、“无干扰性”、“可部署性”三维评价，支持打分、评语与自动给出采纳建议；同时支持对同一 CVE 的多候选策略进行按维度排序，形成偏好数据。
4. **可视化分析与对比**：基于离线汇总文件展示不同方法/评测器在三维维度上的总体表现，并提供样例钻取以支撑定性分析。
5. **发布与回滚（原型定义）**：将发布视为“输出可落地策略并记录采纳决策”，回滚视为“撤销/更改历史决策并保留审计轨迹”。

非功能性需求

为满足论文实验与原型验证，系统还需满足：

- **可用性**：界面清晰、流程可视，评审操作成本低；支持容器化一键启动。
- **可追溯与可审计**：所有评分、排序与决策均需落盘留痕，能够复现评审过程与结果。
- **可扩展性**：方法（method）、评测器（judge）、维度（dimension）等应尽量数据驱动，便于追加新模型或新实验输出。
- **轻量化**：原型阶段采用 CSV/JSON 作为主要存储介质，降低部署复杂度，并为后续迁移至数据库预留空间。

4.2 系统设计

总体架构

系统采用“前端展示层—后端服务层—数据与证据层—智能体管线层”的分层架构，前后端通过 REST API 交互。图 4-1 给出逻辑架构与数据流。

系统逻辑架构（示意）

前端（Vue/ElementPlus/ECharts） ↔ 后端 API（Flask） ↔ 数据/证据（CSV/JSON/Markdown）
↓ 交互式管线编排：检索 → 规划 → 推理 → 策略 → 评审 → 优化
↓ 质量评估：三维评分 + 候选排序 + 统计仪表盘
↓ 发布/回滚：采纳决策与审计留痕

图 4-1：面向云原生权限提升漏洞防控系统原型总体架构

Human-in-the-loop 设计原则

本原型工具将“人在回路”作为核心设计约束：系统不追求完全自动化替代人工，而是把专家判断放在关键节点，并以留痕机制确保可解释、可追溯。主要体现在：

- **证据先行**：生成与评审都以可查看的证据为前提。
- **可编辑会话产物**：管线分阶段产物支持人工修改并回写，形成“生成—校正—再生成”的闭环。
- **人最终决策 + 全量留痕**：系统给出建议，人类确认决策；评分、排序、评语与决策可复盘。

模块划分与职责

结合代码实现，系统可划分为五类核心模块（与后续实现小节对应）：

- **核心模块（展示与编排）**：前端提供多视图入口与交互控件（表格、表单、图表、会话面板等），后端提供统一 API 网关与轻量存储。
- **证据与知识库服务**：负责读取漏洞证据（结构化 CSV、CVE Markdown）与多模型聚合结果，并为管线检索阶段提供缓存目录与索引。
- **策略生成与优化服务**：以会话为单位编排分阶段执行，产出包括检索证据、推理过程、结构化策略、评审意见与优化后的策略。
- **质量评价与审阅**：提供三维评分、候选排序、统计仪表盘与样例钻取；并将人工评审数据持久化为可追溯记录。
- **策略发布与回滚**：在原型阶段，将策略“发布”抽象为采纳决策与策略版本的落盘归档；“回滚”抽象为对历史决策的撤销/更改，并保留全量审计轨迹。

数据与接口设计

原型系统遵循“数据驱动”原则：系统输入与输出均以可读的文件格式呈现，并由后端 API 统一封装。主要数据对象包括：

- **漏洞证据数据**：包含 CVE ID、第三方应用、漏洞摘要、风险等级等字段的结构化表（CSV），以及 CVE 原文（Markdown）。
- **候选策略数据**：对每个 CVE 生成的策略候选集合（JSON），用于排序评审与对比分析。

- **人工评分记录：**针对 CVE 或策略的三维评分与评语（CSV），用于审计与统计。
- **排序历史记录：**针对某个 CVE、某种方法、某个维度的候选策略排序结果（CSV），用于偏好数据沉淀。
- **匿名化映射：**候选策略 ID 与其来源元信息的映射（JSON），用于盲审与追溯之间的折中。

接口层采用 REST 风格，按功能聚类为：证据读取接口、评分接口、排序接口、分析接口与交互式管线接口。该设计一方面保证前端页面的解耦与可扩展，另一方面便于在原型阶段快速迭代与复现实验。

4.3 系统实现与演示

系统原型工具位于 ui/ 目录下，采用前后端分离实现：

- **后端：**基于 Flask 实现 REST API，并对 /api/* 路径开放跨域访问；采用 CSV/JSON/Markdown 文件作为主要存储与证据载体。
- **前端：**基于 Vue3 + Vite 实现单页应用（SPA），使用 ElementPlus 构建交互控件，使用 ECharts 绘制统计图表（如热力图）。
- **部署方式：**支持 docker-compose 一键启动前后端服务，也支持本地脚本启动以便开发调试。

为便于说明，后续从核心模块开始，对证据与知识库、策略生成与优化、质量评价与审阅，以及发布与回滚的原型实现分别展开。

4.3.1 核心模块

核心模块负责把“证据浏览—生成/优化—评分/排序—决策留痕”组织为一套可操作的界面流程。前端以多视图入口承载不同任务（证据浏览、人工评分、候选排序、仪表盘、交互式管线），后端提供统一的 API 与轻量存储以支撑页面交互。

在存储上，原型采用“结构化字段 + 追加写入”方式记录评分、排序与决策，使交互过程可追溯、可复现，并便于后续迁移至数据库。

4.3.2 证据与知识库服务

证据与知识库服务提供两类输入：一是漏洞证据（结构化清单与 CVE 原文），二是多方法/多模型生成产物（用于对比分析与评测）。后端将其统一封

装为可调用接口，前端以“可阅读证据”的形式呈现给评审者；交互式管线会把检索结果落入会话缓存，以支撑复现与审计。

4.3.3 策略生成与优化服务

策略生成与优化服务以会话为单位编排端到端流程，并将每次生成的中间产物纳入会话上下文以便追溯。原型将流程抽象为“检索—推理—生成—评审—优化”五类阶段；其中评审者可对任一阶段产物进行修改并回写，使后续阶段在“人工校正”后继续推进，体现 human-in-the-loop 的闭环特征。策略最终以结构化形式输出，便于阅读、评审与版本化管理。

4.3.4 质量评价与审阅

三维评价体系与打分机制

为支撑策略质量的统一度量，系统原型采用三维评价体系：

- **有效性 (Effectiveness, E)**：策略对权限提升攻击路径的覆盖程度与防护有效性。
- **无干扰性 (Non-interference, N)**：策略对正常业务的影响程度，越不干扰得分越高。
- **可部署性 (Deployability, D)**：策略在云原生环境中的落地成本、可操作性与运维复杂度。

评分采用 0 ~ 10 的量表，并给出加权总分：

$$S = 0.50 \cdot E + 0.35 \cdot N + 0.15 \cdot D.$$

总分映射为三档建议（采纳/带监控采纳/拒绝），并由评审者结合业务背景做最终确认；评语用于记录关键依据，形成可复盘的审阅结论（human-in-the-loop）。

候选策略排序与偏好数据沉淀

系统支持对同一 CVE 的多候选策略进行按维度排序，并将排序历史持久化为偏好数据。为降低来源偏见，候选可匿名化展示；需要审计时可通过映射追溯来源。

统计仪表盘与样例钻取

系统提供基于离线汇总文件的对比仪表盘，用于观察不同方法在三维维度上的整体表现，并支持样例钻取以辅助解释与复盘。

4.3.5 策略发布与回滚

原型阶段的发布定义

原型阶段“发布”不直接执行到真实集群，而是将发布定义为：选定策略版本、给出采纳决策、并完成留痕归档。该定义强调 **human-in-the-loop**：发布是评审确认驱动的决策过程，而非单次自动动作。

回滚与版本化留痕

“回滚”体现为对历史决策的撤销或更改：当证据变化、业务反馈提示干扰、或更优版本出现时，系统允许重新评审并提交新的记录。通过追加写入保留决策链路，从而支持追溯与复现。

演示流程（端到端）

原型系统的端到端演示可概括为：证据浏览与核验；会话化生成与人工修订；评分/排序形成审阅结论；发布决策与留痕回滚。

5 实验研究

5.1 研究问题

本章实验旨在验证所提方法在权限提升漏洞实时防控场景下的有效性与适用性，具体围绕以下研究问题展开：

RQ1: 在权限提升漏洞的临时防控场景下, 本文提出的方法不同步骤能否在有效性 (E)、无干扰性 (I) 与可部署性 (D) 三个维度上相对消融配置提供一致性增益?

RQ2: 本文工作流的相对优势是否对不同生成模型与评审模型稳健, 是否存在系统性的能力分层或偏好差异?

RQ3: 在多评审模型参与的排名式评价中, 不同评审模型给出的相对次序是否具有足够的一致性以支撑跨评审聚合与反馈闭环?

RQ4: 对比现有静态扫描工具, 本文方法能否为具体 CVE 的临时防控提供实质性的处置价值突破?

RQ5: 本文方法在令牌消耗与经济成本上的表现如何, 在给定资源预算下存在何种成本-收益折衷?

5.2 实验设置

实验以 55 个权限提升漏洞实例为对象, 在统一的提示与输出结构规范下对五个大模型生成候选策略集合 Π_v 。表5-1 给出本实验所采用的五个大语言模型的基本信息。为验证关键模块的贡献, 设置三项消融: 移除检索增强 (w/o rag)、移除思维链分阶段推导 (w/o cot)、移除基于评审反馈的迭代优化 (w/o feedback), 其余流程保持一致。

评审阶段将上述五个模型作为评审模型, 对 Π_v 进行排名式评价。评价指标采用三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$, 分别对应有效性、无干扰

表 5-1: 实验采用的大语言模型基本信息

Tab. 5-1: Basic information of large language models used in experiments

模型代号	开发机构	发布日期	参数规模
Qwen3-Max	阿里云	2025 年 9 月	未公开
Deepseekv3.1	深度求索	2025 年 9 月	670B
GPT-5	OpenAI	2025 年 6 月	未公开
Grok4	xAI	2025 年 7 月	未公开
GLM-4.5	智谱 AI	2025 年 7 月	355B

性与可部署性。为降低不同评审模型评分尺度差异带来的影响，本文以相对排序作为主要统计口径。

为回答有关端到端令牌消耗与大模型调用成本的问题，本文在生成与评审阶段记录每次调用的输入与输出的令牌数，并按漏洞样本（CVE）聚合统计端到端令牌消耗；对于包含反馈迭代的配置，累计其多轮调用的令牌消耗。

为对比传统静态安全工具，选取 `kubelinter`、`kubescape`、`kubesec` 和 `SLI-Kube` 作为基线，输出风险/合规报告后人工评估其对策略生成的启发性，例如能否指向与漏洞路径相关的高风险配置、是否给出可落地的修正建议等。

5.3 RQ1：防控技术 workflow 整体优势

本章结果部分按研究问题组织。除非特别说明，统计结果均在 55 个漏洞样本上汇总，评审侧采用排名式结果作为主要口径。

5.3.1 三维度总体表现

图5-1 给出不同方法配置在 *E/I/D* 三个维度上的平均排名式结果。整体上，全量工作流（all）在有效性维度保持领先，同时在不干扰性与可部署性维度呈现出与有效性不同的取舍结构。为刻画样本层面的差异分布，图5-2 进一步给出 all 配置在三个维度上的热力图总览。

5.3.2 有效性维度的消融贡献与稳定性

为更直接回答 workflow 设置对漏洞路径覆盖的贡献，本文以有效性维度作为核心观察对象。图5-3 给出跨评审汇总后的方法配置平均排名，全量方法在

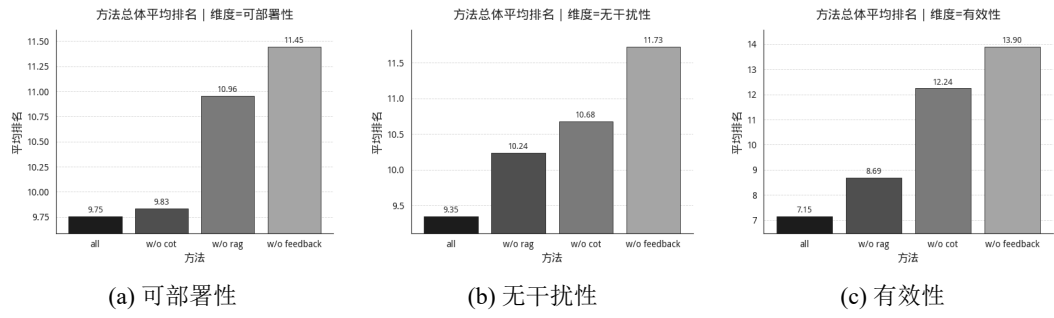


图 5-1: 不同方法配置在三个维度上的平均排名式评分结果（跨评审者汇总，三维度并列）。

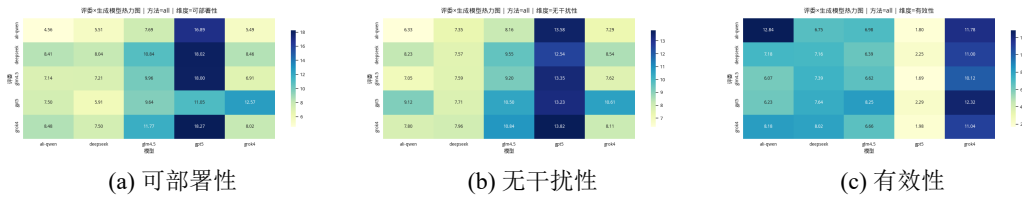


图 5-2: 全量方法配置（all）下三个维度的热力图总览（三维度并列）。

有效性上整体领先。图5-4 与图5-5 分别从评审聚合与生成聚合两个视角展示消融差异，用于检验提升是否对评审模型与生成模型稳健。

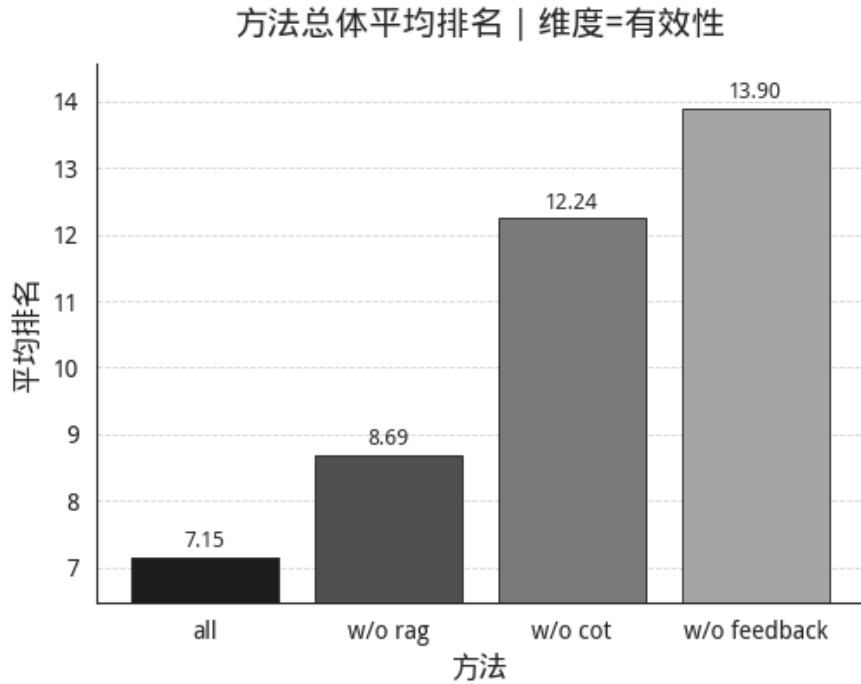


图 5-3: 有效性维度下，不同方法配置的平均排名式评分结果（跨评审者汇总）。

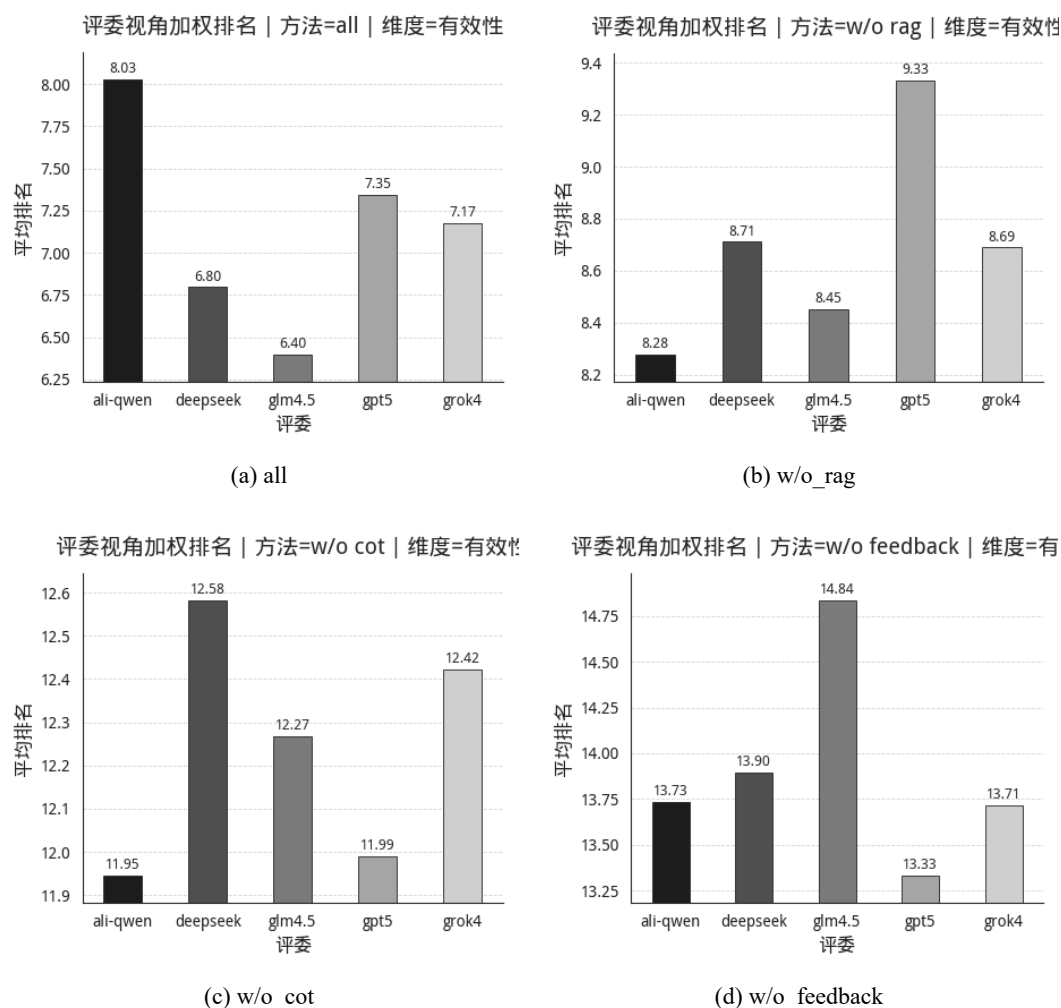


图 5-4: 有效性维度下, 按评审模型聚合的消融对比 (2×2 网格)。

5.3.3 样本层面差异与离群

为检验提升是否在样本层面保持一致, 并识别离群漏洞, 图5-6 给出四种方法配置在有效性维度上的热力图对照。

5.4 RQ2: 跨模型稳健性

5.4.1 生成模型分层

图5-8 给出不同生成模型在有效性维度的平均排名式结果, 用于刻画生成模型能力分层现象。图5-9 与图5-10 进一步从生成模型视角展开方法配置贡献结构, 以验证 all 的相对优势是否在不同生成模型条件下保持一致。

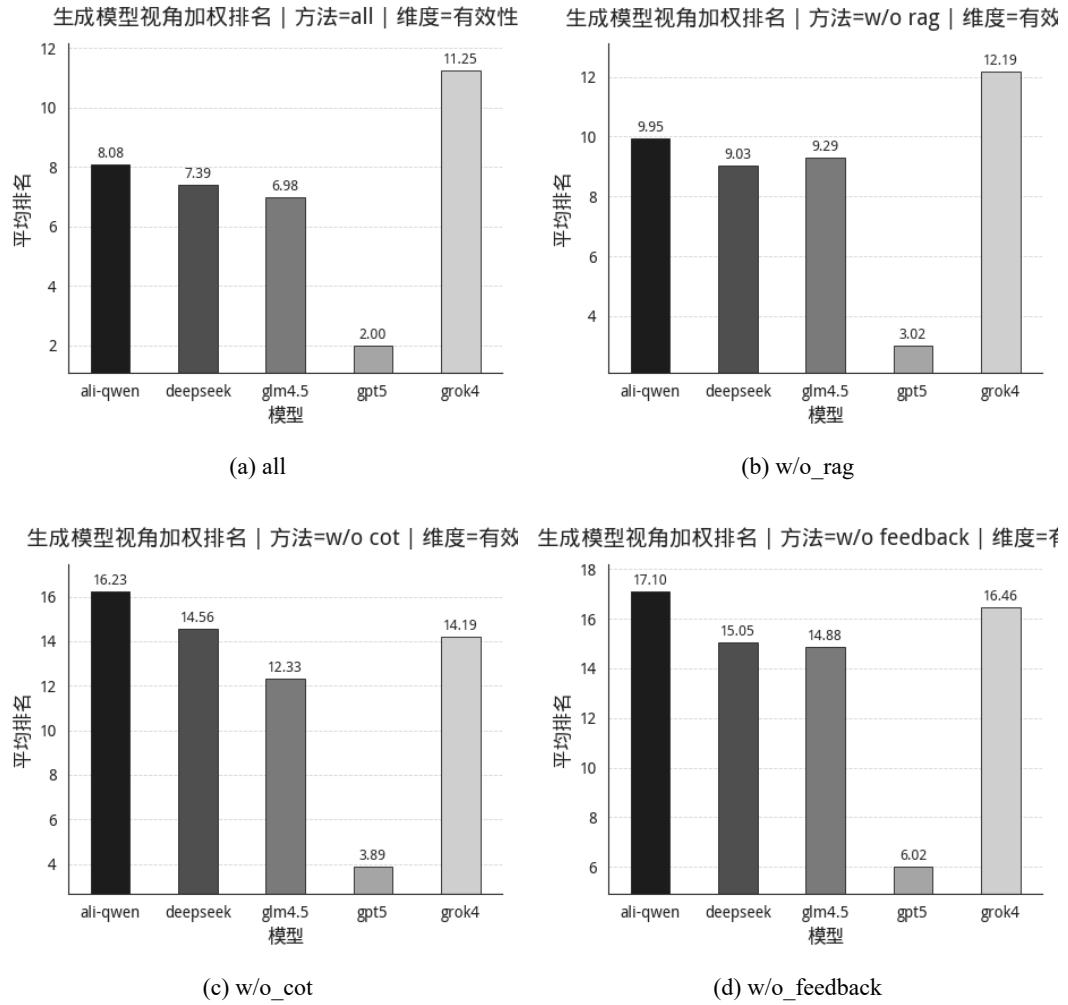


图 5-5: 有效性维度下, 按生成模型聚合的消融对比 (2×2 网格)。

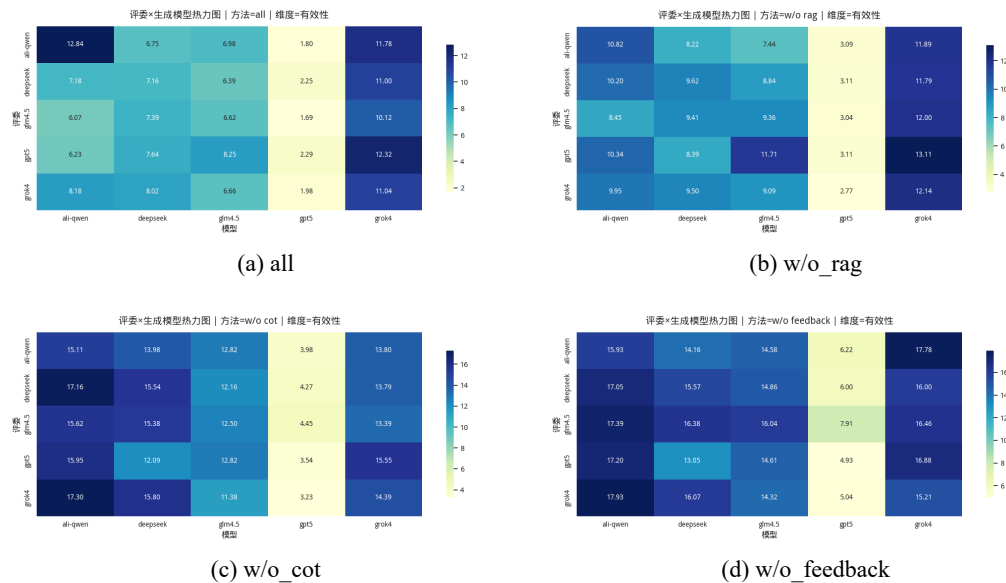


图 5-6: 有效性维度下, 四种方法配置的热力图对照 (2×2 网格)。

TODO: 待补图 (RQ1)

建议图型: all 相对各消融的胜率 (55 个 CVE 上 all 排名优于对照的比例) 与排名差值分布 (箱线图)。

建议数据口径: 按 CVE 聚合, 维度分别为 $E/I/D$; 评审侧可取 5 个评审模型的平均秩或多数表决秩。

图 5-7: 全量工作流相对消融设置的胜率与优势幅度统计 (待补图)。

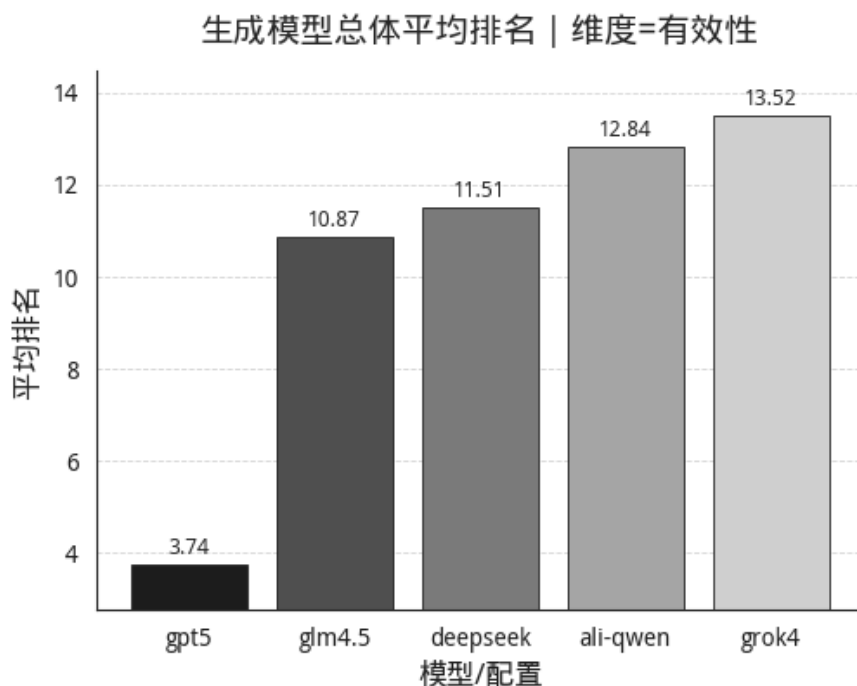


图 5-8: 有效性维度下, 不同生成模型的平均排名式评分结果 (跨评审者汇总)。

5.4.2 评审模型与三维权衡

图5-11 展示不同评审模型下的单维度排名式结果, 反映评审模型之间的尺度差异与偏好结构。图5-12 则从生成模型角度给出三维度总体表现, 用于观察生成模型在 $E/I/D$ 上的系统性取舍。

5.4.3 模型内容对其能力评估

为评估不同生成模型在策略生成过程中的稳定性与输出质量, 本文统计了各模型在生成阶段触发重试的次数。重试机制主要针对三类失败情形: 结构化输出的 JSON 格式错误 (通常因字段缺失、括号不匹配或编码问题)、生成内容的语法错误 (包括 YAML 语法错误、逻辑矛盾或不可解析的配置片段), 以及内容违规 (触发模型供应商的安全策略或内容审核机制)。表5-2 给出五个生成模型在 55 个漏洞样本全流程中的累计重试次数统计。

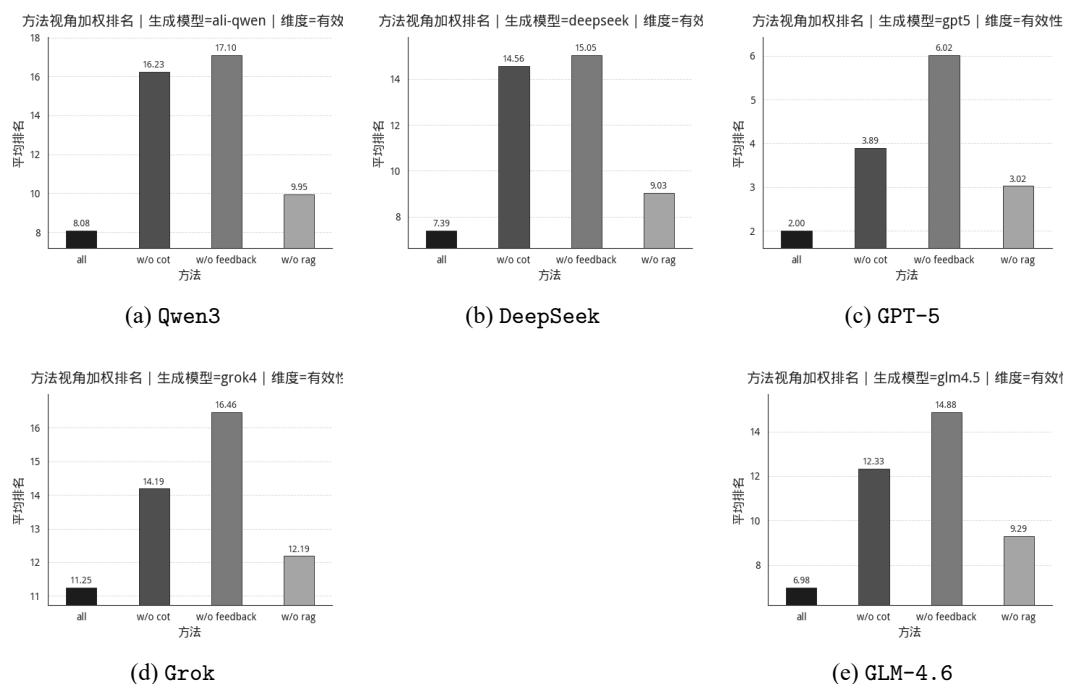


图 5-9: 有效性维度下, 不同生成模型的工作流贡献结构对照 (3+2 网格)。

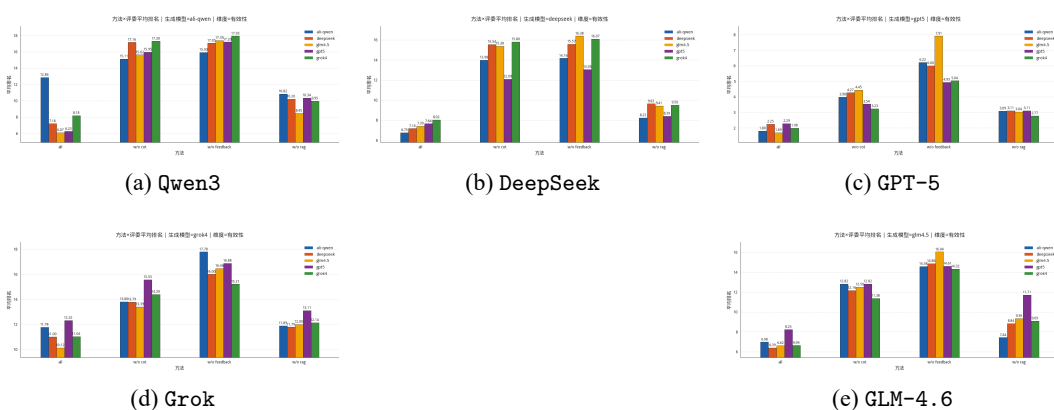


图 5-10: 有效性维度下, 不同生成模型的分组条形图对照 (3+2 网格)。

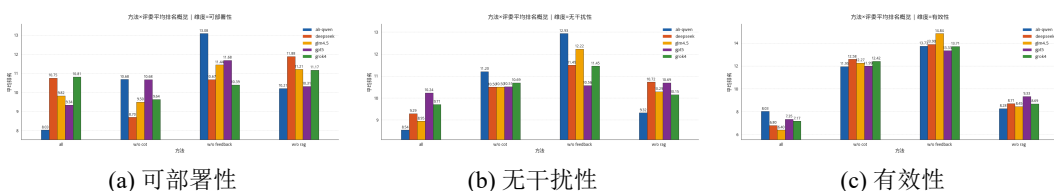


图 5-11: 不同评审模型下的单维度排名式评分结果 (三维度并列)。

整体上, GPT-5 在三类错误上均无重试记录, 表现出较高的生成鲁棒性; Grok 和 DeepSeek 的重试次数较少, 输出稳定性良好; Qwen3 与 GLM-4.5 的重试次数相对较高, 尤其在 JSON 格式错误上, GLM-4.5 达到 23 次, 反映其在结构化输出约束下的遵循能力有待提升。语法错误维度上, GLM-4.5 同样

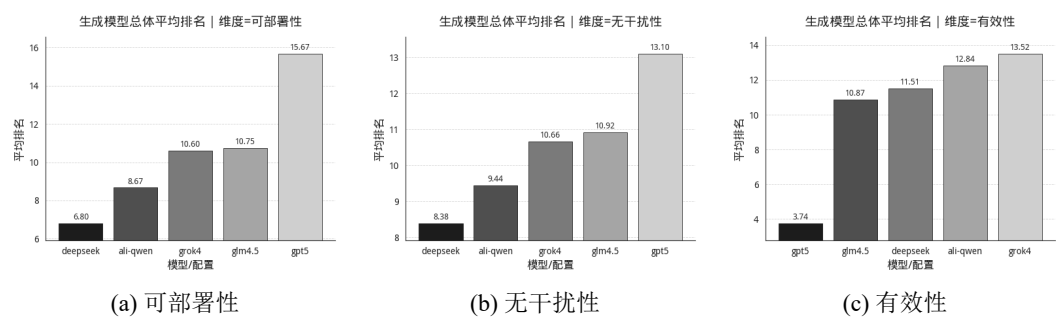


图 5-12: 不同生成模型在三个维度上的平均排名式评分结果（跨评审者汇总，三维度并列）。

表现出明显劣势，共计 10 次重试，而其余模型均在 5 次以下。值得注意的是，GLM-4.5 是唯一触发内容违规的模型（10 次），可能与其内容审核策略对高危漏洞描述配中的安全关键词过度敏感相关。该统计为模型选型与多智能体协同策略提供了量化参考：在资源受限或对输出稳定性敏感的场景下，优先选用 GPT-5、Grok 或 DeepSeek；而对于需要多样性候选集的场景，可将 Qwen3 与 GLM-4.5 纳入候选池，但需配置更宽松的重试预算与后处理容错机制。

表 5-2: 生成模型在策略生成过程中的重试次数统计
Tab. 5-2: Statistics of retry attempts during strategy generation across LLMs

错误类型	DeepSeek3.1	GLM-4.5	GPT-5	Grok4	Qwen3
JSON 格式错误	5	23	0	5	13
语法错误	3	10	0	0	5
内容违规	0	10	0	0	0
合计	8	43	0	5	18

5.5 RQ3: 评价一致性与可信度

图5-13 给出有效性维度下按评审模型拆分的汇总结果，图5-14 进一步展示评审偏好结构。为将一致性讨论落到可计算的量化指标，本文计划补充评审间相关性统计与分布图，用于度量不同评审模型在候选排序上的一致性程度。

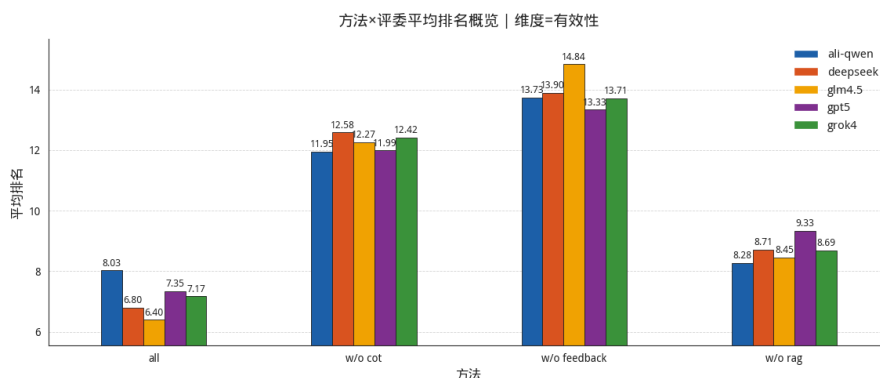


图 5-13: 有效性维度下, 不同评审模型的排名式评分汇总。

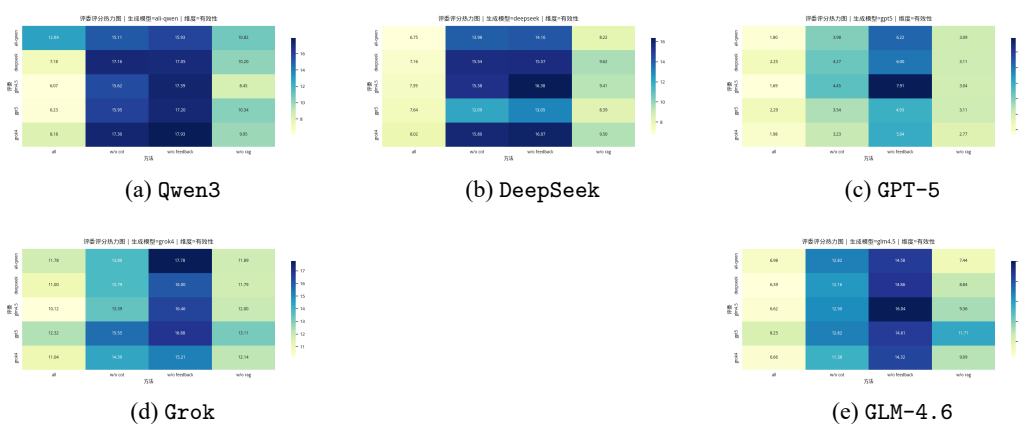


图 5-14: 有效性维度下, 不同评审模型的偏好结构热力图对照 (3+2 网格)。

TODO: 待补图 (RQ3)

建议图型: 评审模型两两一致性热力图 (Kendall's τ 或 Spearman ρ), 分别对 $E/I/D$ 计算。

建议口径: 对每个 CVE 的候选排序, 先得到评审间相关, 再在 55 个 CVE 上取均值或中位数。

图 5-15: 评审模型两两一致性统计热力图 (待补图)。

TODO: 待补图 (RQ3)

建议图型: 每个 CVE 的一致性分数分布 (箱线图/直方图), 并标注一致性最低的若干样本用于案例解释。

图 5-16: 按漏洞样本统计的评审一致性分布 (待补图)。

5.6 RQ4: 静态扫描的处置价值

为评估静态扫描工具对临时防控决策的支撑程度, 本文选取 kubescape、kubesecl 与 kubelinter 作为代表性基线, 并纳入 SLI-Kube 作为对照, 在同一批 Kubernetes 清单上对报告条目按统一风险类型口径归并统计, 结果如表5-3所示。静态工具的报告更集中于资源基线与通用加固项, 适合用于日常治理

风险类型	Kubescape	Kubesecc	Kubelinter	SLI-Kube
资源配置缺失	110	47	73	0
非 Root 运行	54	53	51	0
根文件系统非只读	39	34	59	0
网络隔离	132	0	0	15
特权与提权	39	0	12	0
主机级暴露	30	0	9	3
身份与密钥	192	58	0	2
系统调用隔离	0	58	0	1
探针配置	92	0	15	0

表 5-3: 四种静态扫描工具在统一风险类型上的报告数量统计

与基线巡检；但从 CVE 处置视角看，报告通常难以直接指向漏洞路径与触发前置条件，也难以形成包含验证与回滚边界的可执行动作序列，因此对补丁窗口期的直接支撑作用较弱。

5.7 RQ5：端到端资源消耗与效率评估（令牌消耗）

令牌消耗（token usage）作为衡量生成式工作流在计算与通信资源上开销的代理量，对评估方法在补丁窗口期及受限部署环境中的可用性具有实际意义。本节旨在以严谨的计量口径刻画不同方法配置下的资源消耗结构，并通过成本-收益分析判断在给定资源预算下的最优折衷。

度量与分析设计如下：

在实现层面，本文记录的令牌数不包括人工复核的人工工时成本，也未直接折算为云服务计费（可按厂商计费模型另行转换）。下列图表与表格为基于上述口径的结果占位，后续将以实测统计替换占位内容。

表 5-4: 不同数据集与大模型下的平均 Token 消耗对比
Tab. 5-4: Comparison of average token usage across datasets and LLMs

方法	单个 CVE 的平均 Token 输出消耗					
	Qwen3- max	DeepSeek	GLM-4.5	GPT-5	Grok-4	均值
All	2382.88	4606.87	2902.54	8874.71	2631.29	4279.65
w/o CoT	1713.28	4048.21	2507.39	7571.79	2025.02	3573.14
w/o Feedback	1554.30	3578.10	2478.98	6547.50	1859.59	3203.69
w/o RAG	2063.49	4359.79	2557.12	8411.06	2393.32	3956.95
均值	1928.74	4148.24	2611.51	7851.26	2227.31	3753.36
每千耗费 (元)	0.012	0.003	0.002	0.07	0.06	—
输出花费均值 (元)	0.03	0.014	0.006	0.62	0.158	—

6 案例分析

为验证本文提出的智能安全策略生成框架在真实云原生权限提升漏洞场景下的有效性，本章选取若干典型 CVE 案例进行深入分析。每个案例包括**漏洞复现**、**策略生成**与**三维质量验证**三个阶段：首先在可控环境中重现漏洞攻击路径并确认风险；随后使用本文方法自动生成候选安全策略；最后从有效性（E）、无干扰性（I）、可部署性（D）三个维度对生成策略进行实证评估，以验证策略能否真实阻断攻击、是否引入业务干扰以及部署难度是否可控。

6.1 案例选择与实验设计

6.1.1 CVE 案例选择标准

为保证案例的代表性与可复现性，本文依据以下标准筛选案例 CVE：

（1）风险等级与影响范围：优先选择 CVSS 评分 ≥ 5.0 的高危漏洞，且在 Kubernetes 生态中具有广泛影响（如 Kubernetes 核心组件漏洞或高使用率的第三方应用漏洞）。

（2）成因类型覆盖：确保所选案例覆盖本文提出的两级成因分类框架中的主要类别，包括配置缺陷（如过度权限配置、网络隔离不足）、安全逻辑缺陷（如访问控制缺陷、身份验证绕过）、敏感信息泄露与代码注入等，以全面检验方法的适用性。

（3）可复现性：漏洞具备公开的攻击 PoC（Proof of Concept）或详尽的技术细节，可在隔离环境中安全复现；同时受影响版本与修复版本明确，便于对比验证。

（4）防控价值：漏洞场景具有典型的临时防控需求（如修复补丁尚未发布、业务无法立即升级或需要多层防御），使得智能生成策略具有实际应用价值。

6.1.2 实验环境与复现流程

（1）环境搭建。所有漏洞复现与策略验证在隔离的 Kubernetes 测试集群中进行，集群配置如下：

- Kubernetes 版本:根据各 CVE 受影响版本动态切换(使用 Kind 或 Minikube

快速创建不同版本集群)；

- 节点规模：3 节点集群（1 个控制平面节点 + 2 个工作节点），每个节点 4 核 CPU / 8GB 内存；
- 网络插件与安全工具：根据案例需求部署 Calico (NetworkPolicy)、OPA/-Gatekeeper、Kyverno、Falco 等；
- 受影响应用：在集群中部署具有已知漏洞的应用版本（如特定版本的 Argo CD、Cilium 等）。

(2) 漏洞复现流程。对每个 CVE，按以下步骤进行复现：

1. **基线环境准备：**部署受影响版本的目标组件，并创建模拟的业务负载与用户身份（如低权限 ServiceAccount）；
2. **攻击执行：**根据公开 PoC 或技术分析，模拟攻击者执行权限提升攻击，记录攻击步骤与观测到的异常行为；
3. **影响确认：**验证攻击是否成功实现权限提升（如获取集群管理员权限、访问受限资源、执行特权操作等）；

(3) 策略生成与验证。复现成功后，使用本文方法生成安全策略并进行三维评价：

1. **安全上下文构建 (RAG)：**将 CVE 公告、受影响组件文档、攻击日志与社区讨论输入向量知识库，检索与漏洞相关的证据；
2. **策略生成 (CoT + Feedback)：**基于 RAG 上下文与漏洞分析结果，生成候选策略并进行评审反馈优化；
3. **策略部署与复测：**在测试集群中部署生成的安全策略（如 NetworkPolicy、RBAC 规则、Gatekeeper 策略等），重新执行攻击验证策略是否有效阻断；
4. **三维质量评价：**
 - **有效性 (E)：**策略能否成功阻断原攻击路径，是否存在绕过可能；
 - **无干扰性 (I)：**策略对正常业务流量的影响（如误报率、性能开销、运维复杂度）；
 - **可部署性 (D)：**策略配置的完整性与可执行性，是否包含清晰的验证与回滚方案。

6.2 案例一：CVE-2022-24768 内部权限问题

6.2.1 漏洞详解与复现

(1) 漏洞背景

Argo CD 是 Kubernetes 生态中广泛使用的声明式 GitOps 持续交付工具。其典型部署形态为：Argo CD 控制平面以相对高权限身份（对集群资源拥有创建/更新/删除能力）持续对齐 Git 仓库中的“期望状态”。该架构提升交付一致性的同时，也带来显著的安全挑战：一旦普通用户能够影响 Argo CD 的控制决策，便可能借助控制平面权限实现权限边界放大。

CVE 描述：CVE-2022-24768 为 Argo CD 的不当访问控制（Improper Access Control）漏洞。所有未修复版本（自 1.0.0 起）均存在风险，0.5.x 与 0.8.x 系列中包含该问题的受限版本。攻击者需要满足以下前置条件之一：(i) 对某个 Application 的源 Git/Helm 仓库具备 push 权限；或 (ii) 在 Argo CD 中对该 Application 具备 sync 与 override 权限。满足条件后，攻击者可通过构造/修改仓库内容或应用配置，诱导 Argo CD 以其控制平面权限执行超出攻击者原始 RBAC 边界的操作，进而潜在提升至 admin 级别能力。官方已在 Argo CD 2.1.14、2.2.8、2.3.2 中发布补丁。缓解措施可降低风险但不能替代升级。

主要成因分类：安全逻辑缺陷 / 访问控制缺陷（已授权用户的权限放大）。
攻击链条描述：其抽象攻击链可概括为：

1. **合法身份阶段：**攻击者以普通 Argo CD 用户身份登录；
2. **配置影响阶段：**凭借对 Application 的 sync+override 或对源码仓库的 push 权限，注入恶意资源清单；
3. **控制平面滥用阶段：**Argo CD 按 GitOps 同步逻辑将恶意资源应用到集群；
4. **权限提升阶段：**借助被创建的高权限 RBAC 绑定（如 ClusterRoleBinding）获得集群级管理员能力。

(2) 漏洞复现

本节给出可复现的实验流程：通过构造一个包含“正常资源 + 恶意 RBAC 绑定”的 Git 仓库，并为低权用户配置 sync+override 权限，使其能够触发 Argo CD 同步，从而在集群中创建管理员级绑定实现提权。

环境配置：

- Kubernetes 版本：vX.Y.Z (kind/minikube 均可，本文复现以可部署 Argo

CD 的版本为准，具体版本号以实验环境为准)；

- 受影响组件：Argo CD v2.3.1 (漏洞版本，补丁版本为 2.3.2/2.2.8/2.1.14)；
- 测试负载：
 - Git 仓库：https://github.com/Ivanbeethoven/bad_repo；
 - 其中包含 `app.yaml` (正常资源，如 `nginx`) 与 `hack-admin.yaml` (恶意 `ClusterRoleBinding`，将用户 `bob` 绑定到管理员角色)。

攻击步骤：

本文复现按“部署漏洞版本 → 创建宽松 `AppProject` → 授予低权用户 `sync+override` → 创建指向恶意仓库的 `Application` → 触发同步 → 验证权限变化”的流程进行。为避免正文过长，关键命令与清单见附录B。

攻击结果：综合上述验证，可观察到“权限边界放大”的典型迹象：

- 低权用户 `bob` 原本仅被授予 `applications sync/override` (对特定项目范围内应用)；
- 通过影响 Git 期望状态或应用同步动作，最终间接在集群中创建了高权限 RBAC 绑定；
- 其效果表现为 `kubectl auth can-i '*' '*' --as bob` 条件下返回 `yes`。

```
r2cn-ubuntu-01 → ~ kubectl describe clusterrolebinding bob-admin-binding
Name:          bob-admin-binding
Labels:        app.kubernetes.io/instance=bob-app
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name:  cluster-admin
Subjects:
  Kind  Name  Namespace
  ----  ---  -
  User  bob
```

图 6-1: 漏洞复现实验结果示意 (验证权限变化)

6.2.2 策略部署

使用本文方法生成的最佳安全策略如下 (经多智能体评估体系选出)。该策略遵循“纵深防御 (Defense in Depth)”，强调限制配置可达性、限制配置表达力、限制权限来源，并将“升级补丁”作为必要但非唯一手段。

策略概述：

- 防控层次 (**layer**)：身份与权限 / 策略执行 / 准入控制 / 审计与治理
- 防控方法 (**method**)：Kubernetes RBAC 收敛 + Argo CD RBAC 加固 +

图 6-2: 漏洞复现实验结果示意 (验证是否有命令空间级别权限)

- 使用工具: Kubernetes RBAC、Argo CD RBAC (Casbin)、OPA/Gatekeeper (或 Kyverno)、Kubernetes Audit (以及 SIEM/告警系统)

- **Kubernetes RBAC (集群层)**: 收敛 AppProject 的创建/更新/删除权限，仅平台管理员可写；
- **Argo CD RBAC (应用层)**: `policy.default=role:readonly`，并显式隔离 `projects/repositories/clusters/exec` 等高风险能力；
- **准入控制 (策略层)**: 在 AppProject 创建/更新时校验策略表达，禁止通配符放权、强制项目域前缀、禁止授予高危资源写权限；
- **审计与基线 (治理层)**: 将关键配置纳入 Git 基线与审计告警，降低权限漂移风险。

6.2.3 三因素分析

- Kubernetes RBAC 限制 AppProject 写入，降低“放开 sourceRepos/destinations/whitelist”导致的爆炸半径；

- Argo CD RBAC 默认只读并隔离高危资源写权限，使低权用户难以获得可用于放大控制面的关键操作集合；
- 准入控制对策略表达施加硬约束，防止隐蔽通配符放权与权限漂移；
- 身份治理约束授权来源，降低组注入或错误映射造成的隐式提权。

在复现实验中，若上述策略生效，则攻击者即使拥有 `sync` 权限，也难以通过修改 `AppProject`/仓库/策略来诱导 Argo CD 创建集群级高权限绑定，攻击路径被显著收敛。

(2) 无干扰性 (Interference) 该策略以“最小必要变更”为原则，主要影响集中在管理面与配置流程：

- 对业务同步的直接影响可控：保留项目内正常的 `sync` 能力，仅收紧 `override`、高危资源写权限与危险策略表达；
- 准入控制建议先 `audit` 再 `enforce`，可显著降低误拦截对业务的冲击；
- 额外运维开销主要来自 RBAC 精细化与准入规则维护，但可通过模板化与 GitOps 基线降低长期成本。

(3) 可部署性 (Deployability) 该策略依赖的能力均为云原生常用组件，具有较强可落地性：

- Kubernetes RBAC 与 Argo CD RBAC 为原生能力，配置可版本化并可审计；
- Gatekeeper/Kyverno 为成熟的准入控制方案，可按项目逐步推广；

6.3 案例二：CVE-2025-2241 (Hive ClusterProvision 敏感信息泄漏)

6.3.1 漏洞详解与复现

(1) 漏洞背景

Hive 是 Red Hat Multicluster Engine (MCE) 与 Advanced Cluster Management (ACM) 中的集群生命周期管理组件，常用于在多集群场景下自动化创建与托管 OpenShift/Kubernetes 集群。在 vSphere 场景中，Hive 需要使用 vCenter 凭据完成集群的基础设施供应与后续管理。

CVE 描述： CVE-2025-2241 指出 Hive 在完成 vSphere 集群供应 (`provisioning`) 后，会将 vCenter 凭据暴露在 `ClusterProvision` 对象中。由于 `ClusterProvision` 属于自定义资源 (CR)，拥有其读取权限的用户可直接

从该对象中提取敏感凭据，即使该用户并不具备对 Kubernetes Secret 的访问权限。该问题会导致未授权 vCenter 访问、基础设施与集群管理风险，并可能进一步演化为权限提升。

CVSS 信息：CNA (Red Hat) 给出的向量为 CVSS:3.1/AV:N/AC:H/PR:L, 基准分 8.2 (High)。其含义可理解为：攻击者需要一定前置权限 (PR:L) 和较高攻击复杂度 (AC:H)，但一旦满足条件，影响范围跨越安全边界 (S:C)，并造成高机密性与完整性影响 (C:H/I:H)。

主要成因分类：敏感信息泄漏 / 配置与对象生命周期缺陷 (“Secret 旁路”：敏感数据出现在可读的 CR 状态字段中)。

攻击链条描述：该漏洞可抽象为 “Secret 隔离被 CR 可读性绕过” 的链条：

1. **权限前置阶段：**攻击者拥有对 ClusterProvision 的 get/list/watch 权限 (通常来自聚合角色 view/edit 或错误的绑定)；
2. **数据旁路阶段：**Hive 在供应流程中将 vCenter 凭据写入 ClusterProvision (常见位于 status 字段)；
3. **凭据提取阶段：**攻击者通过读取 CR YAML/JSON 直接获得凭据；
4. **后续风险阶段：**利用 vCenter 凭据对虚拟化平台执行未授权操作，进而影响集群节点与工作负载，造成横向移动或权限提升。

```
r2cn-ubuntu-01 → ~ kubectl auth can-i get secrets -n leak-demo --as=system:serviceaccount:leak-demo:cp-reader
no
r2cn-ubuntu-01 → ~ kubectl auth can-i list clusterprovisions.hive.openshift.io \
-n leak-demo \
--as=system:serviceaccount:leak-demo:cp-reader
yes
```

图 6-3: CVE-2025-2241: vCenter 凭据通过 ClusterProvision 暴露的风险示意

(2) 漏洞复现

本案例复现目标是验证：无 Secret 访问权限但可读 ClusterProvision 的用户，是否可从对象中提取到 vCenter 凭据。由于该漏洞涉及敏感凭据泄露，本文仅给出受控测试环境的复现要点，详细 YAML 与命令序列见附录 C。

环境配置：

- 测试集群：OpenShift 或 Kubernetes (部署 ACM/MCE 与 Hive，具体版本以厂商公告与实验环境为准)；
- 基础设施：vSphere 环境与可用 vCenter 账号 (最小权限、测试专用)；

- 权限模型：至少一个“只读用户/角色”，具备 ClusterProvision 读取权限但不具备 Secret 读取权限。

攻击步骤（摘要）：

1. 创建 vSphere 集群供应配置（Hive ClusterDeployment 等相关对象），触发供应流程；
2. 使用低权限但可读 ClusterProvision 的身份读取对象并导出 YAML；
3. 在输出中定位 vCenter 用户名/密码等敏感字段（例如 `status.vCenter.credentials.*` 或同类结构）。

攻击结果：若在 ClusterProvision 的可读字段中发现明文凭据，则说明“敏感信息通过 CR 状态旁路泄露”成立；该风险不依赖 Secret 读取权限，属于权限边界外泄。

6.3.2 策略部署

使用本文方法生成的最佳安全策略如下（经多智能体评估体系选出）。与“纯升级”不同，本案例强调：**敏感信息泄漏的治理核心在于读权限面（read surface）与暴露窗口（exposure window）管理。**

策略概述：

- **防控层次（layer）：**身份与权限 / 审计与告警 / 资源生命周期治理 / 准入策略（审计型）
- **防控方法（method）：**RBAC 收敛 + 读取审计与告警 + 供应完成后的对象清理（缩短暴露窗口）+ vCenter 凭据轮换与强化 + “敏感字段守护”检测
- **使用工具：**Kubernetes/OpenShift RBAC、Kubernetes Audit、CronJob/自动化清理、Gatekeeper 或 Kyverno（建议以 dryrun/Audit 方式用于检测）

策略配置与部署步骤：为避免正文过长，本案例的复现详细步骤、RBAC 清单、审计策略片段、自动化清理示例与 Gatekeeper/Kyverno 检测规则完整代码见附录C。

6.3.3 三因素分析

（1）有效性（Effectiveness）

- **RBAC 收敛**可直接降低“非必要主体读取 ClusterProvision”的概率，

是最关键的风险抑制手段；

- **审计与告警**可发现异常读取行为，并在凭据轮换后用于验证风险是否存在；
- **对象清理/自动化**可缩短凭据暴露窗口，降低“读取即泄露”的可利用时间；
- **准入策略（审计型）**可帮助在配置漂移或版本回退时及时发现敏感字段再次出现。

（2）无干扰性（Interference）

- RBAC 收敛主要影响运维/排障可见性，需要通过“按需、限时、可审计”的授权流程替代全局可读；
- 审计与告警对业务无直接影响，但会增加日志量与告警治理成本；
- 自动化清理需确保不会破坏 Hive 的后续对账/回收流程，建议先在预生产验证并以标签/状态条件精确匹配删除对象。

（3）可部署性（Deployability）

- RBAC 与审计为平台原生能力，可在多集群场景快速模板化推广；
- 清理与轮换为运维流程问题，建议与 GitOps/流水线集成（供应完成后打标签 → 定时清理）；
- 准入策略建议从 `dryrun/Audit` 起步，作为检测与治理辅助，而非强制拦截控制器写入 `status`（避免影响供应）。

6.4 案例三：CVE-2023-30512（CubeFS CSI 插件集群权限提升）

6.4.1 漏洞详解与复现

（1）漏洞背景

CubeFS 是面向云原生场景的分布式存储系统，常通过 CSI（Container Storage Interface）插件与 Kubernetes 集成。在 Kubernetes 中，CSI 节点插件通常以 DaemonSet 形式在每个节点上运行（例如 `cfs-csi-node`），控制器组件以 Deployment 形式运行（例如 `cfs-csi-controller`）。这类组件需要与 Kubernetes API 交互，但其权限应遵循最小权限原则。

CVE 描述：CVE-2023-30512 的核心问题是 CubeFS CSI 插件的服务账号（如 `cfs-csi-service-account`）被绑定到了包含**过度权限**的集群级角色

主要成因分类：配置缺陷 / 过度授权（Over-privileged RBAC）。

1. **身份前置阶段**：CSI 组件使用默认或被误配置的 `ServiceAccount` 运行；
2. **权限放大阶段**：`ServiceAccount` 被绑定到含 `secrets` 读取能力的 `ClusterRole`；
3. **控制获取阶段**：攻击者控制某节点上的 CSI Pod（例如 `cfs-csi-node`）并获得其运行时上下文；
4. **数据获取阶段**：攻击者以该 `ServiceAccount` 身份读取集群 `Secret`，从而获取高价值凭据（管理员 `Token`、云凭据等），实现从“节点/Pod 控制”向“集群控制”的升级。

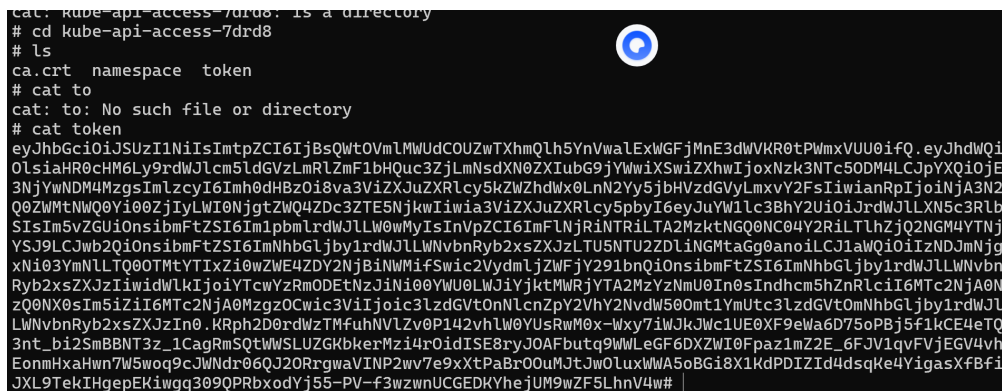


图 6-4: CVE-2023-30512: 攻击成功证据 (基于 CSI ServiceAccount 的越权访问结果)

(2) 漏洞复现（验证性复现）

环境配置 (摘要):

- 部署受影响版本的 CubeFS CSI（通过 Helm 或 YAML 部署，命名空间以实际为准）；
- 明确 CSI 使用的 ServiceAccount（如 `cfs-csi-service-account`）以及绑定的 ClusterRole/ClusterRoleBinding。

复现步骤（摘要）：

1. 盘点并导出 CSI 的 RBAC 绑定，确认 ClusterRole 规则中包含 `resources: ["secrets"]` 且 `verbs` 包含 `get/list/watch`；
2. 以该 ServiceAccount 身份执行权限验证（`kubectl auth can-i`），确认其具备跨命名空间读取 Secret 的能力；
3. 按“最小权限 + 准入防回归 + 审计检测”部署策略后，重复第 2 步验证权限应为 `no`。

```
r2cn-ubuntu-01 → ~ curl -k -H "Authorization: Bearer $TOKEN" $API_SERVER/api/v1/
{
  "kind": "APIResourceList",
  "groupVersion": "v1",
  "resources": [
    {
      "name": "bindings",
      "singularName": "binding",
      "namespaced": true,
      "kind": "Binding",
      "verbs": [
        "create"
      ]
    },
    {
      "name": "componentstatuses",
      "singularName": "componentstatus",
      "namespaced": false,
      "kind": "ComponentStatus",
      "verbs": [
        "get",
        "list"
      ],
      "shortNames": [
        "cs"
      ]
    },
    {
      "name": "configmaps",
      "singularName": "configmap",
      "namespaced": true,
      "kind": "ConfigMap",
      "verbs": [
        "create",
        "delete",
        "deletecollection",
        "get",
        "list",
        "patch",
        "update",

```

图 6-5: CVE-2023-30512：攻击成功证据（Secrets 读取能力验证/提权结果截图）

攻击结果（摘要）：若服务账号具备集群范围 Secret 读取权限，则攻击者一旦控制 CSI Pod/节点，存在获取高价值凭据并进一步获得集群控制的风险。

6.4.2 策略部署

本案例的治理重点是**移除不必要的 Secrets 读取权限并防止权限回归**：对系统组件而言，升级补丁固然重要，但“RBAC 基线治理 + 准入策略 + 审计检测”能够在补丁不可立即应用或配置漂移场景下提供纵深防护。

策略概述：

- **防控层次 (layer)：** 身份与权限 / 策略执行 / 审计与检测 / 网络收敛 / 秘密治理
- **防控方法 (method)：** RBAC 最小化与清理 + ServiceAccount 硬化（必要时禁用自动挂载令牌）+ OPA/Gatekeeper 准入防回归（禁止创建含 secrets 读取动词的 ClusterRole）+ 审计检测（对 secrets 枚举告警）+ 网络出站收敛（仅允许必要目的地）
- **使用工具：** Kubernetes RBAC、OPA/Gatekeeper（或 Kyverno）、Kubernetes Audit（以及 SIEM/告警系统）、NetworkPolicy

策略配置与部署步骤： 为避免正文过长，RBAC 最小化示例、准入控制规则、审计策略片段与验证命令见附录D。

6.4.3 三因素分析

（1）有效性（Effectiveness）

- **RBAC 最小化**直接移除攻击链关键条件（secrets 读取能力），可从根源降低风险；
- **准入防回归**可阻断后续“误配置/回滚/安装脚本重复引入过度授权”的问题；
- **审计检测**可在异常访问出现时快速发现并触发凭据轮换等止血动作。

（2）无干扰性（Interference）

- RBAC 收敛可能影响 CSI 的部分诊断/自愈路径，应在预生产环境验证 PVC/PV 生命周期与 attach/detach 流程；
- 准入策略若采用 deny 强制模式，需避免误拦截平台必需组件的合法权限；建议先 dryrun/audit 运行并基于白名单迭代；
- 审计与检测会增加日志量与告警治理成本，但通常不影响业务链路。

（3）可部署性（Deployability）

- RBAC 与审计为平台原生能力，可在多集群场景模板化推广；

- 准入策略与网络收敛属于“长期基线治理”，可通过 GitOps 与策略即代码持续交付；
- 建议提供回滚与灰度路径：先在测试/预生产验证，再逐步推广到生产。

6.5 案例四：CVE-2020-4062 Conjur OSS Helm Chart 数据库网络暴露

6.5.1 漏洞详解与复现

(1) 漏洞背景

Conjur OSS 是云原生场景下常用的机密管理系统，属于 Secrets Management 系统。在典型部署中，Conjur Server 负责鉴权与策略校验，后端数据库 PostgreSQL 用于持久化存储策略、审计与机密相关数据。Kubernetes 默认网络模型在多数容器网络接口 CNI 配置下呈扁平互通状态，Pod 间通信默认允许。因此，对于数据库等敏感组件，需要显式配置 NetworkPolicy 等隔离策略，形成默认拒绝策略 Default Deny，以控制最小暴露面。

CVE 描述：CVE-2020-4062 影响 Conjur OSS Helm Chart 2.0.0 之前版本。旧版 Chart 默认未提供对 Postgres 的访问控制策略，例如缺失 NetworkPolicy，导致数据库服务在集群内对任意 Pod 可达。即使数据库未通过 NodePort 或 LoadBalancer 暴露到公网，攻击者一旦获得集群内任意工作负载的执行上下文，仍可能横向移动并直连数据库，绕过应用层安全控制，从而造成机密数据泄露、策略被篡改与审计记录被破坏等后果。

严重等级：Critical。

主要成因分类：配置缺陷 / 网络访问控制缺失 Missing NetworkPolicy。

攻击链条描述：该漏洞可抽象为集群内横向移动与直连数据库绕过应用层安全的链条：

1. **初始立足点：**攻击者控制集群内任意 Pod，例如通过业务漏洞或错误授权获得容器执行能力；
2. **侦察发现：**在目标命名空间中发现 Postgres Service 与 5432 端口；
3. **网络直连：**由于缺失 NetworkPolicy，攻击者可从任意 Pod 直连 Postgres；
4. **影响扩展：**一旦数据库凭据或数据密钥被获取，常见来源包括过宽 RBAC、配置泄露或备份泄露，风险可进一步扩展为机密窃取、权限篡改与审计删除。

(2) 漏洞复现（验证性复现）

本案例复现以**网络暴露面验证**为主：证明在旧版 Chart 部署下，Postgres 在集群内可被非 Conjur 组件访问，随后通过部署 NetworkPolicy 将该访问面收敛。考虑到该场景涉及数据库直连与敏感数据风险，本文不展开可被滥用的数据库操作细节，完整的环境准备与策略清单见附录E。

环境配置（摘要）：

- 测试集群：Kubernetes Minikube 与 Helm v3；
- 受影响组件：Conjur OSS Helm Chart < 2.0.0；
- 网络前提：集群容器网络接口 CNI 支持 NetworkPolicy，否则策略无法生效。

复现步骤（摘要）：

1. 在独立命名空间部署旧版 Conjur OSS Chart，并确认 Postgres Service 端口 5432 存在；
2. 证明该命名空间中不存在限制 Postgres 入站的 NetworkPolicy；
3. 进行受控连通性验证：从非 Conjur 组件工作负载发起到 Postgres 5432 的 TCP 探测，用于证明集群内可达；
4. 部署默认拒绝并仅允许 Conjur Server 访问 Postgres 的 NetworkPolicy，复测应不可达。

攻击结果（摘要）：若未隔离 Postgres，攻击者一旦获得集群内任意 Pod 的执行上下文，即存在横向移动并直连数据库的机会，形成从业务容器到机密管理后端的跨域突破。

6.5.2 策略部署

该漏洞的**根本修复**是升级 Conjur OSS Helm Chart 至 2.0.0 或更高版本，新版本默认包含更严格的网络策略。在无法立即升级的场景下，本案例给出以 NetworkPolicy 为核心的缓解策略：采用默认拒绝策略 Default Deny 收敛数据库入站访问面，仅允许 Conjur Server 访问 Postgres 5432 端口，并辅以命名空间隔离与持续审计，降低配置漂移风险。

策略概述：

- 防控层次 **layer**：网络隔离 / 身份与权限 / 审计与治理
- 防控方法 **method**：NetworkPolicy 默认拒绝与精确放通，仅允许 Conjur Server → Postgres 5432，配合命名空间隔离、RBAC 收敛与基线检测

- **使用工具：**Kubernetes NetworkPolicy、Kubernetes RBAC、审计与告警系统

策略配置与部署步骤：为避免正文过长，NetworkPolicy 示例、验证命令与注意事项见附录E。

6.5.3 三因素分析

(1) 有效性 (Effectiveness)

- NetworkPolicy 能直接收敛数据库暴露面，将任意 Pod 可达降低为仅 Conjur Server 可达，从网络层阻断横向直连路径；
- 命名空间隔离与 RBAC 收敛可降低数据库凭据被读取/滥用的概率，与网络隔离形成互补；
- 基线检测可在 Helm 回滚、配置漂移时及时发现 NetworkPolicy 缺失或规则被弱化。

(2) 无干扰性 (Interference)

- NetworkPolicy 变更可能影响 Conjur 组件间通信与运维排障，需要在预生产验证并保留回滚方案；
- 若集群 CNI 不支持 NetworkPolicy，则策略无效；此时应优先通过命名空间隔离或选择支持策略的 CNI 解决；
- 精确放通依赖正确的标签选择器，标签不一致会导致误拦截，需要与 Chart 模板保持一致。

(3) 可部署性 (Deployability)

- NetworkPolicy 属于原生资源，易于 GitOps 化与多集群复制；
- 规则相对稳定，数据库服务相对固定，维护成本较低；
- 建议将 NetworkPolicy 纳入 Helm values 或部署流水线的强制检查项，避免遗漏或回归。

7 结论与展望

7.1 结论

本文面向云原生权限提升漏洞的快速缓解需求，提出了以“证据可追溯、结构化产物、质量闭环”为核心的策略生成框架，主要结论如下：

- 提出 `layer/method/actions` 结构化策略表示与 JSON 结构规范约束，结合证据来源标注，使生成结果可检查、可回溯、可复核。
- 构建开源应用知识库与多源检索增强的安全上下文，分块、去重与元数据保留提高了证据覆盖度与复现性。
- 设计“RAG+CoT+Feedback”分阶段流程，将漏洞分析、控制手段选型与策略生成显式化，并通过评审反馈迭代优化候选策略，降低噪声与误报。
- 以门槛评分与三维质量向量（有效性、无干扰性、可部署性）对候选策略进行规则化评价与排序，实验表明各模块均对交付质量有显著贡献，生成式方法较静态工具更能产出可落地的缓解序列。

7.2 展望

未来工作可从以下方向展开：

- **数据与覆盖面：**扩展漏洞与应用样本，纳入供应链依赖、运行时观测数据，增强对链式攻击场景的适配能力。
- **验证与部署自动化：**将生成策略在隔离环境中自动验证（语法检查、仿真与回归测试），缩短从生成到上线的闭环时间。
- **评审增强与人机协同：**引入更细粒度的风险标注与可解释性输出，优化评审提示与可视化，降低安全工程师的复核成本。
- **与现有工具集成：**将生成式流程与静态/动态检测工具、CI/CD 流水线结合，实现持续化的策略生成、验证与发布。

参考文献

- [1] Kubernetes. Kubernetes documentation. Available online: <https://kubernetes.io/docs/home/>, 2024. Accessed: November 18, 2024.
- [2] Cloud Native Computing Foundation. Annual survey 2022. Available online: <https://www.cncf.io/reports/cncf-annual-survey-2022/>, 2022. Accessed: November 18, 2024.
- [3] Alibaba Cloud. Alibaba container service: From serverless containers to serverless kubernetes. Available online: https://www.alibabacloud.com/blog/from-serverless-containers-to-serverless-kubernetes_596533, 2020. Accessed: November 18, 2024.
- [4] David Ferraiolo, Janet Cugini, D Richard Kuhn, et al. Role-based access control (rbac): Features and motivations. In *Proceedings of the 11th Annual Computer Security Applications Conference*, pages 241–48, 1995.
- [5] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [6] Greg O’Shea. Redundant access rights. *Computers & Security*, 14(4):323–348, 1995. ISSN 0167-4048.
- [7] Nanzi Yang, Wenbo Shen, Jinku Li, Xunqi Liu, Xin Guo, and Jianfeng Ma. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS 2023)*, page 3048–3062, 2023. ISBN 9798400700507. doi: 10.1145/3576915.3623121.
- [8] Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Transactions on Software Engineering and Methodology*, 32(4):1–36, May 2023. ISSN 1049-331X.

- [9] Nicholas Pecka, Lotfi Ben Othmane, and Altaz Valani. Privilege escalation attack scenarios on the devops pipeline within a kubernetes environment. In *Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering(ICSSP)*, page 45–49, 2022. ISBN 9781450396745. doi: 10.1145/3529320.3529325.
- [10] Chang-ai Sun, Xiaoyang Han, and Yufei Gong. Kubeguard: A systematic permission-oriented risk detection approach for kubernetes applications. In *2025 IEEE International Conference on Web Services (ICWS)*, pages 855–865, 2025. doi: 10.1109/ICWS67624.2025.00111.
- [11] Aqua Security. Trivy: Universal scanner for vulnerabilities, misconfigurations, and sbom. GitHub repository, 2025. URL <https://github.com/aquasecurity/trivy>. Accessed 2026-01-06.
- [12] Anchore. Grype: Vulnerability scanner for container images and filesystems. GitHub repository, 2025. URL <https://github.com/anchore/grype>. Accessed 2026-01-06.
- [13] Anchore. Syft: Cli tool and library for generating software bill of materials. GitHub repository, 2025. URL <https://github.com/anchore/syft>. Accessed 2026-01-06.
- [14] StackRox. Kubelinter: Static analysis for kubernetes yaml files. GitHub repository, 2025. URL <https://github.com/stackrox/kube-linter>. Accessed 2026-01-06.
- [15] ARMO. Kubescape: Kubernetes risk analysis, compliance, and scanning. GitHub repository, 2025. URL <https://github.com/kubescape/kubescape>. Accessed 2026-01-06.
- [16] Zegl. kube-score: Kubernetes object analysis with recommendations. GitHub repository, 2025. URL <https://github.com/zegl/kube-score>. Accessed 2026-01-06.
- [17] Control Plane. kubesecc: Security risk analysis for kubernetes resources.

- GitHub repository, 2025. URL <https://github.com/controlplaneio/kubesecc>. Accessed 2026-01-06.
- [18] Fairwinds. Polaris: Kubernetes best practices validation. GitHub repository, 2025. URL <https://github.com/FairwindsOps/polaris>. Accessed 2026-01-06.
- [19] Bridgecrew. Checkov: Static analysis for infrastructure as code. GitHub repository, 2025. URL <https://github.com/bridgecrewio/checkov>. Accessed 2026-01-06.
- [20] Yue Gu, Xin Tan, Yuan Zhang, Siyan Gao, and Min Yang. Epscan: Automated detection of excessive rbac permissions in kubernetes applications. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 11–11. IEEE Computer Society, 2024.
- [21] Carmine Cesarano and Roberto Natella. Kubefence: Security hardening of the kubernetes attack surface. *arXiv preprint arXiv:2504.11126*, 2025.

附录 A 单位

以下内容可放在附录之内：

- (1) 正文内过于冗长的公式推导；
- (2) 方便他人阅读所需的辅助性数学工具或表格；
- (3) 重复性数据和图表；
- (4) 论文使用的主要符号的意义和单位；
- (5) 程序说明和程序全文；
- (6) 企业应用证明；
- (7) 项目鉴定报告；
- (8) 获奖成果证书；
- (9) 设计图纸；
- (10) 程序源代码；
- (11) 论文发表；
- (12) 作者简介。

这部分内容可省略。如果省略，删掉此页。

书写格式说明：

标题“附录 A 附录内容名称”样式为字体：黑体，英文用 Times New Roman 字体，居中，加粗，字号：小三，2.41 倍行距，段前 17 磅，段后为 16.5 磅。

附录正文样式为字体宋体小四，英文用 Times New Roman 字体小四，两端对齐书写，段落首行左缩进 2 个字符。1.3 倍行距（段落中有数学表达式时，可根据表达需要设置该段的行距），段前 0.1 行，段后 0.1 行，1.3 倍行距。

示例

表 A-1: 表 A.1 国际单位制的基本单位

量的名称	单位名称	单位符号
长度	米	m
质量	千克（公斤）	kg
时间	秒	s
电流	安〔培〕	A
热力学温度	开〔尔文〕	K
发光强度	坎〔德拉〕	cd

表 A-2: 表 A.2 国家规定的非国际单位制单位

量的名称	单位名称	单位符号	换算关系和说明
时间	分	min	1 min=60 s
	[小] 时	h	1 h=60 min=3600 s
	天(日)	d	1 d=24 h=86400 s
平面角	[角] 秒	"	$1'' = (\pi/648000) \text{ rad}$
	[角] 分	'	$1' = 1^\circ/60 = (\pi/10800) \text{ rad}$
	度	°	$1^\circ = (\pi/180) \text{ rad}$
	度	°	$1^\circ = (\pi/180) \text{ rad}$
旋转速度	转每分	r/min	1 r/min=(1/60) r/s
长度	海里	n mile	1 n mile=1852 m
速度	节	kn	1 kn=1 n mile/h=(1852/3600) m/s (只用于航行)

附录 B CVE-2022-24768 复现步骤与策略清单

本附录给出案例一（CVE-2022-24768）的复现详细步骤与纵深防御策略代码清单，用于复核与工程复现。为降低对生产环境的风险，建议仅在隔离测试集群中执行。

B.1 漏洞复现详细步骤

步骤 1：部署漏洞版本 Argo CD (v2.3.1)

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argocd/v2.3.1/manifests/install.yaml
kubectl rollout status deployment/argocd-server -n argocd
```

步骤 2：获取初始管理员密码并登录 UI/CLI

```
kubectl -n argocd get secret argocd-initial-admin-secret -o
  jsonpath="{.data.password}" | base64 -d && echo
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

步骤 3：创建宽松的 AppProject（用于演示风险）

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: demo-project
  namespace: argocd
spec:
  destinations:
  - namespace: '*'
    server: '*'
  sourceRepos:
  - '*'
```

```
clusterResourceWhitelist:
- group: '*'
  kind: '*'
```

EOF

步骤 4: 创建低权用户 bob, 并授予其对目标 Application 的 sync+override 权限在 argocd-rbac-cm 中添加如下策略（注意缩进）:

```
policy.default: role:readonly
policy.csv: |
    p, role:syncer, applications, sync, demo-project/*,
    ↪ allow
    p, role:syncer, applications, override, demo-project/*,
    ↪ allow
    g, bob, role:syncer
```

并在 argocd-cm 中开启账户:

```
accounts.bob: apiKey, login
```

随后重启 Argo CD Server 使配置生效:

```
kubectl rollout restart deployment/argocd-server -n argocd
```

步骤 5: 创建 Application 指向恶意仓库

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bob-app
  namespace: argocd
spec:
  project: demo-project
  source:
    repoURL:
      ↪ https://github.com/Ivanbeethoven/bad_repo
```



```

        path: .
        targetRevision: HEAD
    destination:
        server: https://kubernetes.default.svc
        namespace: default
    syncPolicy:
        automated:
            prune: true
            selfHeal: true
EOF

```

步骤 6: 模拟 bob 触发同步（或依赖自动同步）

```

argocd login localhost:8080 --username bob --password
↪ <token-or-password>
argocd app sync bob-app --force

```

步骤 7: 验证是否提权成功

```

kubectl auth can-i '*' '*' --as bob

```

若输出为 yes, 说明 bob 获得了集群范围的高权限（该结果用于说明风险，生产环境严禁在真实集群中进行）。

B.2 策略代码清单

B.2.1 Kubernetes RBAC: 收敛 AppProject 的写权限

将 appprojects.argoproj.io 的创建/更新/删除限制在平台管理员组, 租户仅只读（示例中组名可按实际替换）:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
    name: argocd-appproject-edit
rules:
- apiGroups: ["argoproj.io"]

```

```

    resources: ["appprojects"]
    verbs: ["get","list","watch","create","update","patch",
↪      "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
    name: argocd-appproject-readonly
rules:
- apiGroups: ["argoproj.io"]
    resources: ["appprojects"]
    verbs: ["get","list","watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
    name: argocd-appproject-edit-platform-admins
roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: argocd-appproject-edit
subjects:
- kind: Group
    name: argo:platform-admin
    apiGroup: rbac.authorization.k8s.io
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
    name: argocd-appproject-readonly-tenants
roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole

```

```

        name: argocd-appproject-readonly
subjects:
- kind: Group
    name: argo:tenant-readonly
    apiGroup: rbac.authorization.k8s.io

```

B.2.2 Argo CD RBAC: 默认只读 + 高风险能力隔离

建议将 argocd-rbac-cm 纳入 Git 基线管理,并在修改后滚动重启 argocd-server:

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:tenant-readonly, role:readonly

    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow
    p, role:admin, applications, *, *, allow
    p, role:admin, exec, *, *, allow
    p, role:admin, logs, *, *, allow

    p, role:readonly, applications, get, *, allow
    p, role:readonly, repositories, get, *, allow
    p, role:readonly, projects, get, *, allow
    p, role:readonly, clusters, get, *, allow
    p, role:readonly, logs, get, *, allow
    p, role:readonly, exec, *, *, deny
  policy.default: role:readonly

```

B.2.3 准入控制：Gatekeeper（推荐）

目标：在 AppProject 创建/更新时，对 `spec.roles[].policies[]` 进行一致性校验。本策略覆盖三类约束：

- 禁止出现 `*`，`*`，`allow` 的全局通配符放权；
- 要求 policy subject 绑定到项目域（`proj:<project>:<role>` 前缀）；
- 禁止在 project 内授予 `projects/repositories/clusters/exec` 等高危资源写权限。

(1) ConstraintTemplate

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sargocdappprojectpolicies
spec:
  crd:
    spec:
      names:
        kind:
          ↪ K8sArgoCDAppProjectPolicies
      validation:
        openAPIV3Schema:
          properties:
            enforceProjPref_
              ↪ ix:
                type:
                  ↪ boo_
                  ↪ lean
      targets:
        - target: admission.k8s.gatekeeper.sh
          rego: |
            package k8sargocdappprojectpolicies

            trim_space(s) = out {
```

```

        out :=
        ↪ regex.replace("^\\s+|\\s+$",
        ↪ "", s)
    }

```

1) 禁止出现 "..., *, *, allow"

↪ 的全局通配符放权

```

deny[msg] {
    input.review.kind.group ==
    ↪ "argoproj.io"
    input.review.kind.kind ==
    ↪ "AppProject"
    roles := input.review.object.sp
    ↪ ec.roles
    some i
    policies := roles[i].policies
    some j
    policy := policies[j]
    regex.match(".*,\\s*\\*,\\s*\\*
    ↪ ,\\s*allow\\s*$", policy)
    msg := sprintf("AppProject
    ↪ policy contains global
    ↪ wildcard allow: %s",
    ↪ [policy])
}

```

2) 要求 subject 采用

↪ proj:<project>:<role> 前缀（可选开关）

↪

```

deny[msg] {
    input.review.kind.group ==
    ↪ "argoproj.io"

```

```

input.review.kind.kind ==
  ↪ "AppProject"
input.parameters.enforceProjPre
  ↪ fix == true
ap := input.review.object.metad
  ↪ ata.name

roles := input.review.object.sp
  ↪ ec.roles
some i
policies := roles[i].policies
some j
policy := policies[j]

startswith(policy, "p,")
parts := split(policy, ",")
count(parts) >= 2
subject := trim_space(parts[1])
not startswith(subject,
  ↪ sprintf("proj:%s:", [ap]))
msg := sprintf("AppProject
  ↪ policy subject must start
  ↪ with 'proj:%s:': %s", [ap,
  ↪ policy])
}

```

3) 禁止授予

```

  ↪ projects/repositories/clusters/exec
  ↪ 的写权限
deny[msg] {
  input.review.kind.group ==
    ↪ "argoproj.io"

```

```

input.review.kind.kind ==
  ↪ "AppProject"
roles := input.review.object.sp
  ↪ ec.roles
some i
policies := roles[i].policies
some j
policy := policies[j]
regex.match("^p,.*, (projects|re
  ↪ positories|clusters|exec),\
  ↪ \s*(create|update|patch|del
  ↪ ete|\\*),.*, (allow)\\s*$",
  ↪ policy)
msg := sprintf("AppProject
  ↪ policy grants write on
  ↪ global resource: %s",
  ↪ [policy])
}

```

(2) Constraint

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sArgoCDAppProjectPolicies
metadata:
  name: restrict-argocd-appproject-policies
spec:
  match:
    kinds:
      - apiGroups: ["argoproj.io"]
        kinds: ["AppProject"]
  parameters:
    enforceProjPrefix: true

```

B.2.4 准入控制：Kyverno（替代方案）

若集群已使用 Kyverno，可用如下 ClusterPolicy 实现同类校验。建议先将 `validationFailureAction` 设为 `Audit`，稳定后再切换为 `Enforce`。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-argocd-appproject-policies
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: forbid-global-wildcard-allow
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/
                ↪ AppProject
      validate:
        message: "forbid global wildcard allow
        ↪ in AppProject role policies"
        foreach:
          - list: "request.object.spec.roles[].po_
          ↪ lices[]"
            deny:
              conditions:
                any:
```



```

- key: "{{ rege
  ↪ x_match('.*'
  ↪ ,\\s*\\*,\\
  ↪ s*\\*,\\s*a
  ↪ llow\\s*$',
  ↪ element) }}"
      operato
        ↪ r:
        ↪ Equ
        ↪ als
      value:
        ↪ true

- name: enforce-proj-prefix
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/
              ↪ AppProject
      validate:
        message: "policy subject must use
          ↪ proj:<project>:<role> prefix"
        foreach:
          - list: "request.object.spec.roles[].po
            ↪ licies[]"
            deny:
              conditions:
                all:

```

```

- key: "{{
  ↪ regex_match_
  ↪ ('^p,\\s*pr_
  ↪ oj:[^:]+:[^_
  ↪ ,]+,',
  ↪ element) }}"
  operato_
  ↪ r:
  ↪ Equ_
  ↪ als
  value:
  ↪ fal_
  ↪ se

- name: forbid-global-resource-write
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/_
              ↪ AppProject

  validate:
    message: "forbid write on
    ↪ projects/repositories/clusters/exec
    ↪ in AppProject policies"
    foreach:
      - list: "request.object.spec.roles[] .po_
        ↪ licies[]"
        deny:
          conditions:
            any:

```

```
- key: "{{"
  ↪ regex_match_
  ↪ ('^p,.*(pr_
  ↪ ojects|repo_
  ↪ sitories|cl_
  ↪ usters|exec_
  ↪ ),\\s*(crea_
  ↪ te|update|p_
  ↪ atch|delete_
  ↪ |\\*),.*(a_
  ↪ llow)\\s*$',
  ↪ element) }}"
      operato_
        ↪ r:
        ↪ Equ_
        ↪ als
      value:
        ↪ true
```


附录 C CVE-2025-2241 复现步骤与策略清单

本附录给出案例二（CVE-2025-2241）的复现要点与缓解策略清单，用于复核与工程落地。该漏洞属于**敏感信息泄漏**：vCenter 凭据出现在 Hive 的 ClusterProvision 对象可读字段中，导致“无 Secret 权限也可读到凭据”的旁路风险。请仅在隔离测试环境中操作。

C.1 漏洞复现详细步骤（受控测试）

前提条件

- 已部署 ACM/MCE 与 Hive，并启用 vSphere 集群供应能力（具体版本与安装方式依赖环境，以厂商公告与实验环境为准）；
- 至少存在一个“只读身份”，其不具备 secrets 读取权限，但具备 clusterprovisions.hiv 的 get/list/watch 权限（该条件用于验证旁路泄露）。

步骤 1：触发一次 vSphere 集群供应通过 ACM 控制台或 YAML 创建 Hive 的 ClusterDeployment（示例仅展示关键字段，真实环境中密码应引用 Secret；本漏洞的问题在于后续对象中出现明文暴露）：

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: vsphere-cluster
  namespace: your-namespace
spec:
  baseDomain: example.com
  platform:
    vsphere:
      vCenter: vc.example.com
      username: administrator@vsphere.local
      password: your-password
  provisioning:
    installConfigSecretRef:
      name: your-install-config
```

```
imageSetRef:
  name: img4.14.0
```

步骤 2：用“可读 ClusterProvision 但不可读 Secret”的身份读取对象先确认权限面，再读取 ClusterProvision：

```
# 谁能读 clusterprovisions (OpenShift)
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision

# 低权用户自检 (Kubernetes 通用)
kubect1 auth can-i get clusterprovision --as=<user>
↪ --all-namespaces
kubect1 auth can-i get secrets --as=<user> --all-namespaces

# 导出对象 (名称可能带随机后缀, 按实际替换)
kubect1 get clusterprovision -n your-namespace
kubect1 get clusterprovision <name> -n your-namespace -o yaml
```

步骤 3：定位敏感字段在输出中查找类似结构（位置因版本实现而异，示例以 status.vCenter.credentials 为代表）：

```
status:
  vCenter:
    credentials:
      username: ...
      password: ...
```

若凭据以明文形式出现，即满足漏洞复现判据。

C.2 策略代码清单

本节策略以“收敛读取面 + 缩短暴露窗口 + 审计发现与轮换止血”为主线。由于敏感信息出现在控制器写入的 status 字段中，准入控制更适合作为审计/检测手段，不建议强制拦截控制器更新（否则可能影响供应流程）。

C.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)

风险盘点命令 (OCP)

```
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision
oc adm policy who-can watch clusterprovision
```

最小可读权限示例 (仅绑定 Hive 控制器服务账号) 说明: Hive 控制器的具体 ServiceAccount/namespace 需按实际部署确认。

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: hive-clusterprovision-read-min
rules:
- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["get","list","watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: hive-clusterprovision-read-min-binding
subjects:
- kind: ServiceAccount
  name: hive-controllers
  namespace: hive
roleRef:
  kind: ClusterRole
  name: hive-clusterprovision-read-min
  apiGroup: rbac.authorization.k8s.io
```

C.2.2 2) 对读取行为启用审计并联动告警

Kubernetes 审计策略片段 (加载方式依平台而定):

```

apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get","list","watch"]
  resources:
  - group: "hive.openshift.io"
    resources: ["clusterprovisions"]
    omitStages: ["RequestReceived"]

```

建议在 SIEM/日志平台中将 Hive 控制器主体加入白名单，仅对非白名单主体的读取行为告警。

C.2.3 3) 缩短暴露窗口：供应完成后清理 ClusterProvision

手动清理

```

oc get clusterprovision -n <ns>
oc delete clusterprovision -n <ns> <name>

```

自动化清理（示例 CronJob）说明：删除条件建议通过标签或状态字段精确匹配，避免误删；示例以标签 `hive.openshift.io/retain=false` 为条件。

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: cleanup-sa
  namespace: hive
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cleanup-role
  namespace: hive
rules:

```



```

- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["list","get","delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cleanup-rb
  namespace: hive
subjects:
- kind: ServiceAccount
  name: cleanup-sa
roleRef:
  kind: Role
  name: cleanup-role
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-clusterprovision
  namespace: hive
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName:
            ↪ cleanup-sa
          containers:
            - name: cleaner

```

```
image: registry_
↳ .access.red_
↳ hat.com/ubi_
↳ 9/ubi:latest
command: ["/bin_
↳ /sh", "-c"]
args:
- |
    set -eu
```

```
oc delete
→ te
→ clu
→ ste
→ rpr
→ ovi
→ sion
→ -n
→ hive
→ -l
→ hiv
→ e.o
→ pen
→ shi
→ ft.
→ io/
→ ret
→ ain
→ =fa
→ lse
→ --i
→ gno
→ re-
→ not
→ -fo
→ und
```

```
restartPolicy: OnFailure
```

C.2.4 4) vCenter 凭据轮换与强化（止血）

建议在确认泄露后立即轮换 vCenter 凭据，并启用最小权限、MFA、来源 IP 允许列表等约束。轮换完成后更新 Hive 使用的 Secret（示例命令需按实际 Secret 名称/namespace 调整）：

```
oc -n hive create secret generic vcenter-credentials \
  --from-literal=username=<new-user> \
  --from-literal=password=<new-pass> \
  --dry-run=client -o yaml | oc apply -f -
```

C.2.5 5) “敏感字段守护”检测（Gatekeeper/Kyverno, 建议审计模式）

Gatekeeper (dryrun)：检测 ClusterProvision 中是否出现敏感键名（password/token/secret/credential）。

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sclustprovkeysensitive
spec:
  crd:
    spec:
      names:
        kind: K8sClustProvKeySensitive
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sclustprovkeysensitive

        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "password")
        }

        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "token")
        }

        sensitive_key(k) {
```

```

        lk := lower(k)
        contains(lk, "secret")
    }

    sensitive_key(k) {
        lk := lower(k)
        contains(lk, "credential")
    }

# 深度遍历 object, 收集所有键
walk(obj, path, val) {
    is_object(obj)
    some k
    v := obj[k]
    walk(v, array.concat(path, [k]),
        ↪ val)
}

walk(obj, path, val) {
    is_array(obj)
    some i
    v := obj[i]
    walk(v, array.concat(path,
        ↪ [sprintf("%v", [i])]), val)
}

walk(obj, path, obj) {
    not is_object(obj)
    not is_array(obj)
}

deny[msg] {
    input.review.kind.group ==
        ↪ "hive.openshift.io"
    input.review.kind.kind ==
        ↪ "ClusterProvision"
}

```

```

        walk(input.review.object, [], _)
        some k
        _ = input.review.object[k]
        sensitive_key(k)
        msg := sprintf("ClusterProvision
        ↪ contains sensitive key at
        ↪ top-level: %s", [k])
    }

```

```

deny[msg] {
    input.review.kind.group ==
    ↪ "hive.openshift.io"
    input.review.kind.kind ==
    ↪ "ClusterProvision"
    walk(input.review.object, path,
    ↪ _)
    some i
    k := path[i]
    sensitive_key(k)
    msg := sprintf("ClusterProvision
    ↪ contains sensitive key on
    ↪ path %v", [path])
}

```

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sClustProvKeySensitive
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  enforcementAction: dryrun
  match:
    kinds:

```

```
- apiGroups: ["hive.openshift.io"]
  kinds: ["ClusterProvision"]
```

Kyverno (Audit): 对对象文本进行检测，以提示可能存在的敏感键名。
该策略更适用于告警与排查，不建议直接用于阻断控制器更新。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: detect-secret-like-keys
      match:
        any:
          - resources:
              kinds:
                - hive.openshift.io/v1/ClusterProvision
      validate:
        message: ClusterProvision
        ↪ 可能包含敏感字段 password token
        ↪ secret credential。
      deny:
        conditions:
          any:
            - key: "{{ to_string(request.object) }}"
              operator: AnyIn
              value:
                - password
                - token
```

- secret
- credential

附录 D CVE-2023-30512 复现步骤与策略清单

本附录给出案例三（CVE-2023-30512, CubeFS CSI 插件过度授权导致的集群权限提升风险）的**验证性复现与策略清单**。为避免提供可被滥用的细粒度利用操作，复现部分以“权限证据 + 受控权限验证”为主，重点证明 CSI ServiceAccount 被绑定了不必要的 secrets 读取能力；策略部分给出可落地的 RBAC 收敛、准入防回归与审计检测。

D.1 漏洞复现详细步骤（验证性复现）

变量约定（按实际替换）

```
NAMESPACE=<cubeFS-namespace>
SA=cfs-csi-service-account
CLUSTERROLE=cfs-csi-cluster-role
```

如 SA 名称不同，请先定位：

```
kubectl get sa -A | findstr /i "csi cfs cubeFS"
kubectl get clusterrole,clusterrolebinding -A | findstr /i "csi
↪ cfs cubeFS"
```

步骤 1：导出 RBAC 证据（确认 secrets 权限存在）

```
kubectl get clusterrole $CLUSTERROLE -o yaml
kubectl get clusterrolebinding -o yaml | findstr /i "cfs-csi
↪ cubeFS"
```

重点检查 ClusterRole 规则中是否出现：

```
# resources: ["secrets"]
# verbs: ["get","list","watch"]
```

步骤 2：受控权限验证（不执行实际数据提取）

直接验证该 SA 是否具备跨命名空间读取 secrets 的能力

```
kubectl auth can-i get secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

```
kubect1 auth can-i list secrets
```

```
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

```
kubect1 auth can-i watch secrets
```

```
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

权限提升路径（文字图，便于复盘）

CubeFS CSI 安装环境



一旦攻击者控制节点或 CSI Pod => 可滥用该 SA 的身份边界

（风险：读取高价值凭据 -> 演化为集群级权限提升）

D.2 策略代码清单

本节策略遵循“**最小权限 + 防回归 + 可观测**”主线：先移除 CSI 不必要的 `secrets` 读取权限；再用准入策略阻断未来再次引入过度授权；最后启用审计检测以便快速发现异常访问并触发止血流程。

D.2.1 1) RBAC 最小化：移除 ClusterRole 中的 secrets 权限

说明：CSI 需要的资源/动词与具体实现有关，建议在预生产以 PVC/PV 创建、挂载、扩容、attach/detach 等流程回归验证后再推广到生产。以下示例展示“覆盖式”更新 ClusterRole 的方式，明确不包含 `resources: ["secrets"]`。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cfs-csi-cluster-role
  labels:
    app.kubernetes.io/name: cfs-csi
rules:
- apiGroups: [""]
  resources: ["nodes","persistentvolumes","pods"]
  verbs: ["get","list","watch"]
- apiGroups: ["storage.k8s.io"]
  resources: ["volumeattachments","csinodes"]
  verbs: ["get","list","watch"]
# 注意：不应包含 resources:["secrets"]
EOF

# 复测（期望输出 no）
kubectl auth can-i list secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

D.2.2 2) ServiceAccount 硬化（按需评估）

若 CSI 驱动不需要直接调用 Kubernetes API，可评估禁用自动挂载令牌以降低“被攻陷即可滥用身份”的风险：

```
# 将 automountServiceAccountToken 设为 false（需评估 CSI
↪ 功能是否依赖 token）
kubectl -n $NAMESPACE patch serviceaccount $SA --type='merge' -p
↪ '{"automountServiceAccountToken":false}'
```

D.2.3 3) 准入防回归 (OPA Gatekeeper): 禁止创建包含 secrets 读取动词的 ClusterRole

说明：该策略用于防止未来误配置再次引入过度授权。建议先以 dryrun 运行观察影响，再逐步强制。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sdenysecretsrbac
spec:
  crd:
    spec:
      names:
        kind: K8sDenySecretsRBAC
      targets:
        - target: admission.k8s.gatekeeper.sh
          rego: |
            package k8sdenysecretsrbac

            has_secrets_rule(rule) {
              rule.resources[_] == "secrets"
              rule.verbs[_] == "get"
            }

            has_secrets_rule(rule) {
              rule.resources[_] == "secrets"
              rule.verbs[_] == "list"
            }

            has_secrets_rule(rule) {
              rule.resources[_] == "secrets"
              rule.verbs[_] == "watch"
            }
```

```
violation[{"msg": msg}] {
    input.review.kind.kind ==
        ↪ "ClusterRole"
    some i
    rule :=
        ↪ input.review.object.rules[i]
    has_secrets_rule(rule)
    msg := "ClusterRole must not
        ↪ grant get/list/watch on
        ↪ secrets"
}
```

EOF

```
cat <<'EOF' | kubectl apply -f -
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sDenySecretsRBAC
metadata:
    name: deny-clusterrole-secrets-read
spec:
    enforcementAction: dryrun
    match:
        kinds:
        - apiGroups: ["rbac.authorization.k8s.io"]
          kinds: ["ClusterRole"]
```

EOF

D.2.4 4) 审计与检测：对 secrets 枚举行为进行可观测化

Kubernetes 审计策略片段（加载方式依平台而定）：

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
```

```

verbs: ["get","list","watch"]
resources:
- group: ""
  resources: ["secrets"]
omitStages: ["RequestReceived"]

```

建议在 SIEM/日志平台中对来自 CSI ServiceAccount 的 secrets 枚举进行聚合告警，并结合阈值与白名单降低噪音。

D.2.5 5) 网络收敛（可选）：限制 CSI Pod 出站访问面

说明：NetworkPolicy 能降低被攻陷工作负载的横向移动能力，但在不同 CNI 下支持情况不同；同时 API Server 可能位于外部 LB，需按实际环境调整。

```

cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: cfs-csi-egress-lockdown
  namespace: $NAMESPACE
spec:
  podSelector:
    matchLabels:
      app: cfs-csi
  policyTypes: ["Egress"]
  egress:
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
    podSelector:
      matchLabels:
        k8s-app: kube-dns
  ports:

```

```
- protocol: UDP  
  port: 53
```

EOF

附录 E CVE-2020-4062 复现步骤与策略清单

本附录给出案例四（CVE-2020-4062，Conjur OSS Helm Chart 旧版本缺失网络隔离导致 Postgres 在集群内可达）的复现要点与缓解策略清单。考虑到该问题涉及数据库直连带来的敏感数据风险，复现环节仅进行网络暴露面验证，用于确认 5432 端口在集群内的可达性以及网络策略的生效性，不展开数据库操作细节。

E.1 漏洞复现详细步骤：受控测试

前提条件

- 集群 CNI 支持 NetworkPolicy，例如 Calico。
- Helm v3 可用。
- 受影响的 Conjur OSS Helm Chart 版本为 < 2.0.0。

步骤 1：在隔离命名空间部署旧版 Chart，并进行必要的兼容性适配旧版 Chart 可能使用过时 API，例如 apps/v1beta2。在较新 Kubernetes 版本上复现时，可在保持网络策略缺失这一特性不变的前提下，将 Deployment API 适配为 apps/v1 并补齐 selector 字段以完成安装。

步骤 2：确认 Postgres Service 与 5432 端口暴露

NAMESPACE=conjur-vuln

```
kubectl get ns $NAMESPACE
```

```
kubectl -n $NAMESPACE get svc
```

重点确认 Postgres Service（名称以实际为准）与 5432 端口存在

```
kubectl -n $NAMESPACE get svc | findstr /i "postgres 5432
```

```
↪ conjur"
```

步骤 3：确认未配置 NetworkPolicy

```
kubectl -n $NAMESPACE get networkpolicy
```

若不存在任何约束 Postgres 入站的策略，且集群默认允许 Pod 间互通，则可判定数据库对集群内工作负载开放。

步骤 4：连通性验证在不执行数据库操作的前提下，使用 TCP 连通性探测验证非 Conjur 组件工作负载是否能够与 Postgres 的 5432 端口建立连接，以此作为暴露面存在的验证依据。

E.2 策略代码清单

E.2.1 1) 根本修复：升级 Chart

将 Conjur OSS Helm Chart 升级到 2.0.0 或更高版本，使其默认包含更严格的 NetworkPolicy。

E.2.2 2) 缓解措施：NetworkPolicy，默认拒绝并仅放通 Conjur Server 访问 Postgres 5432

下述策略在命名空间内实现两点：其一，默认拒绝 Postgres Pod 的进站流量；其二，仅允许 Conjur Server 访问 Postgres 的 5432 端口。其中标签选择器需与实际部署对象一致，示例使用 `app=conjur-oss` 与 `app=conjur-oss-postgres`，可按实际标签调整。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-default-deny
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
    - Ingress
    ingress: []
EOF
```

```
cat <<'EOF' | kubectl apply -f -
```

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-allow-conjur
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: conjur-oss
      ports:
        - protocol: TCP
          port: 5432
EOF

```

策略验证

```
kubectl -n conjur-vuln get networkpolicy
```

```

# 复测：从非 Conjur 组件工作负载到 Postgres 5432 的连通性应失败
# 从 Conjur Server 到 Postgres 5432 的连通性应成功

```

E.2.3 3) 命名空间隔离与 RBAC 收敛

- 将 Conjur 部署在独立命名空间，并通过 NetworkPolicy 限制跨命名空间访问。
- 收敛对数据库凭据相关 Secret 的读取权限，降低网络可达与凭据可读叠加带来的风险。

致 谢

致谢是作者对该论文的形成作出过贡献的组织或个人予以声明的文字记载，语言要客观、准确、简短。

字数一般不超过 500 字，最多不超过一页。论文作者可以在致谢页对下列方面致谢：

国家科学基金、资助研究工作的奖学金基金，合同单位、资助或支持的企业、组织或个人；

协助完成研究工作和提供便利条件的组织或个人；

在研究工作中提出建议和提供帮助的人；

给予转载和引用权的资料、图片、文献、研究思想和设想的所有者；

其它应感谢的组织或个人。

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2019 年 9 月至 2023 年 6 月	在北京科技大学计算机科学与技术专业攻读学士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

四、在学期间发表的论文

- [1] Sun C, **Han X**, Gong Y. KubeGuard: A Systematic Permission-Oriented Risk Detection Approach for Kubernetes Applications[C]//2025 IEEE International Conference on Web Services (ICWS). IEEE, 2025: 855-865.

盲审说明（正式写作请删除此说明）：

盲审论文需要隐藏掉所有会影响盲审结果的论文作者及其导师的信息，以便论文评阅人能够公正的进行评阅。

当学校要求提供盲审论文时，请按如下方法制作。

1) 对于论文中下列学生和导师信息。请将学生姓名、学生学号、导师姓名，依次全部替换为[本论文作者]、[论文作者学号]、[本论文导师]。论文中上述信息均需要替换，包括作者研究成果等部分的有关信息。

2) 在研究成果中，论文作者发表的文章列表中应隐去所有作者的名字，只标明论文期刊名，级别，发表年份。

3) 盲审论文，请不要填写致谢，致谢页除标题、页眉、页码外请保持空白。

4) 其他会影响盲审结果的信息, 请采用类似方式处理。

5) 提交的盲审论文应为正式论文, 除了上述替换后的信息外, 应为可以评阅的正式论文。

6) 论文封面, 请填写专业名称、论文题目等, 将学号和姓名项目保持空白不填。制作盲审论文时, 论文书脊、封二、题名页请删除。

替换前后将文档分别保存, 以便盲审论文与其他论文分开管理。

盲审样例 (正式写作请删除此样例):

五、在学期间发表的论文:

[1] **第一作者**. 中国矿业. 中文核心期刊.2020.

[2] **第二作者 (导师一作)**. 中国矿业. 中文核心期刊.2020.

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的
研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包
含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其
它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所
做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集

关键词 *	密级 *	中图分类号 *	UDC	论文资助
学位授予单位名称 *		学位授予单位代码 *	学位类别 *	学位级别 *
北京科技大学		10008		
论文题名 *		并列题名		论文语种 *
作者姓名 *			学号 *	
培养单位名称 *		培养单位代码 *	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学院路 30 号	100083
学科专业 *		研究方向 *	学制 *	学位授予年 *
论文提交日期 *				
导师姓名 *			职称 *	
评阅人	答辩委员会主席 *	答辩委员会成员		
电子版论文提交格式文本 () 图像 () 视频 () 音频 () 多媒体 () 其他 () 推荐格式: application/msword; application/pdf				
电子论文出版 (发布) 者		电子论文出版 (发布) 地		权限声明
论文总页数 *				
共 33 项, 其中带 * 为必填数据, 为 22 项。				